ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

# CERTIFICATE

This is to certify that the Cornerstone project work entitled **"STOCK PRICE PREDICTION USING TIME-SERIES FORECASTING MODELS"** is submitted by

| | |
|---|---|
| **R. VENKATA GOPI** | 24B11CS374 |
| **N. VENKATA NATESH** | 24B11CS310 |
| **K. CHARAN TEJA** | 24B11CS223 |
| **V. HARI VEERA VENKAT** | 24B11CS481 |
| **K. KALYAN SANKAR** | 24B11CS208 |

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2025-2026.

**Project Guide**

**J. Lalu Prasad,**

Assistant Professor,

Department of CSE

**Head of the Department**

**Dr.Phani Sridhar**

Associate Professor & HOD,

Department of CSE

# DECLARATION

We hereby declare that the Cornerstone project entitled " STOCK PRICE PREDICTION USING TIME-SERIES FORECASTING MODELS" is original and is submitted to **ADITYA UNIVERSITY** , Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree in Computer Science and Engineering** . We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

**By**


| | |
|---|---|
| **R. VENKATA GOPI** | 24B11CS374 |
| **N. VENKATA NATESH** | 24B11CS310 |
| **K. CHARAN TEJA** | 24B11CS223 |
| **V. HARI VEERA VENKAT** | 24B11CS481 |
| **K. KALYAN SANKAR** | 24B11CS208 |

# ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mr.Lalu Prasad**, **Assistant Professor,** who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. K. Chandra Sekhar, Associate Professor and Dr.Hari Jyothula, Professor, Dr. Siva Satya Prasad P, Assistant Professor,** Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Associate Professor,** for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management,** Aditya University for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the faculty members, lab technicians, non-teaching staff, and our friends who have directly or indirectly supported us in completing this Cornerstone project successfully.

# ABSTRACT

Stock price prediction is an important application of data science and machine learning that helps investors and financial analysts make informed decisions. Due to the dynamic and complex nature of the stock market, accurately forecasting future stock prices remains a challenging task. This project aims to predict stock prices using historical stock market data and compare the performance of different time series forecasting techniques.

The dataset consists of historical stock data containing attributes such as Open, High, Low, Close, and Volume. To achieve accurate forecasting, five models from different categories are implemented: ARIMA and Prophet from statistical forecasting, Support Vector Regression (SVR) and XGBoost from machine learning, and Long Short-Term Memory (LSTM) from deep learning. The data is preprocessed, trained, and tested using each model to generate future stock price predictions. The models are applied to the daily price history of representative Indian and global equities, namely Tata Consultancy Services and Reliance Industries from the NSE, and Alphabet Inc. as a global benchmark.

The performance of the models is evaluated using metrics such as Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE). A comparative analysis is then performed to identify the strengths and limitations of each approach. The results of this study provide insights into the effectiveness of statistical, machine learning, and deep learning models for stock price forecasting and help determine the most suitable technique for time series prediction tasks.

# TABLE OF CONTENTS

**Chapter** **Page No.**

**ABSTRACT** ..... **iii**

**LIST OF FIGURES** ..... **vii**

**LIST OF TABLES** ..... **viii**

**ACRONYMS** ..... **ix**

**1. INTRODUCTION** ..... **1**

1.1 Brief Information about the Project ..... 1

1.2 Problem Statement / Motivation ..... 1

1.3 Objectives of the Project ..... 2

1.4 Scope of the Project ..... 3

**2. SYSTEM ANALYSIS** ..... **4**

2.1 Functional Requirements ..... 4

2.2 Non Functional Requirements ..... 5

2.3 Requirements Specification ..... 6

2.3.1 Hardware Requirements ..... 6

2.3.2 Software Requirements ..... 6

**3. TECHNOLOGY DESCRIPTION** ..... **8**

3.1 Python ..... 8

3.2 Pandas ..... 8

3.3 NumPy ..... 8

3.4 Matplotlib ..... 8

3.5 Scikit-Learn ..... 8
3.6 Statsmodels ..... 9
3.7 Prophet ..... 9
3.8 TensorFlow / Keras ..... 9
3.9 XGBoost ..... 9
3.10 Yahoo Finance ..... 9
3.11 yfinance ..... 9
**4. DATASET DESCRIPTION ..... 10**
4.1 Dataset Source ..... 10
4.2 Dataset Collection Method ..... 11
4.3 Dataset Attributes ..... 12
**5. PROJECT MODULES ..... 14**
5.1 Data Collection Module ..... 14
5.2 Data Preprocessing Module ..... 14
5.3 Forecasting Module ..... 14
5.4 Model Training Module ..... 14
5.5 Evaluation Module ..... 15
5.6 Visualization Module ..... 15
**6. ALGORITHMS USED ..... 16**
6.1 ARIMA ..... 16
6.2 PROPHET ..... 16
6.3 LSTM ..... 17
6.4 XGBOOST ..... 17
6.5 SVR ..... 18

**7. SYSTEM IMPLEMENTATION ..... 19**

7.1 ARIMA Implementation ..... 19

7.2 Prophet Implementation ..... 22

7.3 LSTM Implementation ..... 24

7.4 XGBoost Implementation ..... 26

7.5 SVR Implementation ..... 29

**8. EXECUTION RESULTS ..... 29**

8.1 ARIMA Results ..... 32

8.2 Prophet Results ..... 35

8.3 LSTM Results ..... 38

8.4 XGBoost Results ..... 40

8.5 SVR Results ..... 43

**9. PERFORMANCE EVALUATION ..... 46**

9.1 Evaluation Metrics ..... 46

9.2 Performance Comparison ..... 46

9.3 Analysis of Results ..... 48

9.4 Discussion ..... 48

9.5 Best Model Observation ..... 49

**10. CONCLUSION ..... 50**

**11. FUTURE ENHANCEMENTS ..... 52**

**12. REFERENCES ..... 53**

**13. Paper Publication ..... 54**

# LIST OF FIGURES

**Figure 8.1: Downloaded dataset of Reliance Industries from Yahoo Finance.....32**

**Figure 8.2: ARIMA (5,1,0) model training summary.....33**

**Figure 8.3: Actual versus ARIMA-predicted prices on the test period .....34**

**Figure 8.4: Thirty-day forward forecast produced by the ARIMA model .....35**

**Figure 8.5: Dataset prepared in the Prophet (ds, y) format.....36**

**Figure 8.6: Prophet model training output.....36**

**Figure 8.7: Actual versus Prophet-predicted prices on the test period .....37**

**Figure 8.8: Thirty-day forward forecast and component decomposition of Prophet .....38**

**Figure 8.9: Scaled closing prices and generated input sequences for the LSTM.....38**

**Figure 8.10: LSTM model architecture and training progress .....39**

**Figure 8.11: Actual versus LSTM-predicted prices on the test period.....39**

**Figure 8.12: Thirty-day forward forecast produced by the LSTM model .....40**

**Figure 8.13: Engineered lag-feature table for XGBoost.....41**

**Figure 8.14: XGBoost training output with early stopping .....41**

**Figure 8.15: Actual versus XGBoost-predicted prices on the test period.....42**

**Figure 8.16: Thirty-day forward forecast produced by the XGBoost model.....43**

**Figure 8.17: Scaled lag features prepared for the SVR model .....44**

**Figure 8.18: SVR model training output .....44**

**Figure 8.19: Actual versus SVR-predicted prices on the test period .....45**

**Figure 8.20: Thirty-day forward forecast produced by the SVR model .....45**

**Figure 9.1: Comparison of MAE and RMSE of the five models on RELIANCE.NS.....47**

# LIST OF TABLES

**Table 2.1: Hardware requirements of the system ..... 6**

**Table 2.2: Software requirements of the system ..... 6**

**Table 4.1: Dataset source properties ..... 10**

**Table 4.2: Sample companies used in the study ..... 10**

**Table 4.3: Description of dataset attributes ..... 12**

**Table 4.4: Sample dataset records (RELIANCE.NS, January 2025) ..... 13**

**Table 9.1: Performance comparison of the five models on RELIANCE.NS ..... 47**

**Table 9.2: RMSE of each model across the three sample stocks ..... 47**

# ACRONYMS

- AI — Artificial Intelligence
- ADF — Augmented Dickey-Fuller (test)
- AIC — Akaike Information Criterion
- ANN — Artificial Neural Network
- API — Application Programming Interface
- ARIMA — Auto-Regressive Integrated Moving Average
- BSE — Bombay Stock Exchange
- CSV — Comma Separated Values
- df — DataFrame (pandas object)
- IDE — Integrated Development Environment
- LSTM — Long Short-Term Memory
- MAE — Mean Absolute Error
- MAPE — Mean Absolute Percentage Error
- MSE — Mean Squared Error
- NSE — National Stock Exchange of India
- ReLU — Rectified Linear Unit (activation function)
- RBF — Radial Basis Function (kernel)
- RMSE — Root Mean Square Error
- RNN — Recurrent Neural Network
- RSI — Relative Strength Index
- SGD — Stochastic Gradient Descent
- SVR — Support Vector Regression
- TCS — Tata Consultancy Services

# CHAPTER 1
# INTRODUCTION

## 1.1 Brief Information about the Project

This project, entitled "Stock Price Prediction using Time-Series Forecasting Models", is a comparative study that implements and evaluates five forecasting approaches on historical stock market data, namely ARIMA, Facebook Prophet, Long Short-Term Memory (LSTM) networks, XGBoost and Support Vector Regression (SVR). The historical data is drawn from Yahoo Finance, with the National Stock Exchange (NSE) and the Bombay Stock Exchange (BSE) serving as the underlying markets for the Indian equities considered in the study. Tata Consultancy Services (TCS), Reliance Industries Limited and Alphabet Inc. (GOOGL) are used as representative Indian and global test cases.

Financial markets generate enormous volumes of data every trading day, and the closing price of a stock is the single most closely watched number produced by this activity. If the future movement of this price could be anticipated with reasonable accuracy, investors could manage risk better, analysts could value companies more confidently, and algorithmic trading systems could be designed more intelligently. This project studies how far that anticipation is actually possible using only the historical price record, by building five models from four very different modelling families and measuring them against one another on identical data.

The complete system is developed in Python using the Jupyter Notebook environment. Data is collected programmatically with the yfinance library, prepared using pandas and NumPy, modelled with statsmodels, Prophet, TensorFlow/Keras, XGBoost and scikit-learn, and visualised with Matplotlib. Every model is trained on the same historical window and tested on the same held-out period so that the comparison between them is fair and reproducible.

## 1.2 Problem Statement / Motivation

Stock prices are the result of the interaction of thousands of buyers and sellers reacting to company earnings, interest rates, politics, news flow and market sentiment. Because of this, prices are noisy, non-stationary and frequently regime-shifting, which makes stock price prediction one of the most challenging applications of machine learning and data analytics. A model that works well during

a calm, trending market may fail badly during a sudden correction, and no single technique has ever dominated all others across all market conditions.

The practical problem that motivates this project is that an investor or analyst who wants a price forecast has to choose one modelling approach from a very wide spectrum, ranging from classical statistical equations to deep neural networks, usually without any objective evidence about which approach suits the stock they are studying. Textbooks present each method in isolation, and vendor documentation naturally advertises the strengths of its own library while remaining silent about its weaknesses.

Therefore, there is a clear need for a single, controlled experiment in which several forecasting families are implemented on the same data, tuned with reasonable effort, evaluated with the same error metrics and compared honestly. This project addresses that need. The motivating questions are: how accurately can each family reproduce the held-out price path of an NSE-listed stock and a global stock, how does accuracy decay as the forecast horizon moves from a few days to several weeks, and is the additional complexity of a deep learning model actually repaid by better forecasts on daily equity data.

## 1.3 Objectives of the Project

The main objective of this project is to develop a stock price forecasting system that predicts future prices from historical market data and to rigorously compare five models drawn from four distinct modelling families: a classical statistical model (ARIMA), a decomposition-based model (Prophet), a deep-learning model (LSTM) and two feature-based machine learning models, a gradient-boosted tree ensemble (XGBoost) and a kernel-based method (Support Vector Regression, SVR). Because these families make very different assumptions about how markets behave, comparing them shows which approach best captures the behaviour of a given stock. The specific objectives are:

- To collect historical Open-High-Low-Close-Volume (OHLCV) data for Indian and global equities from Yahoo Finance using the yfinance library, with NSE and BSE as the underlying and cross-check sources for Indian listings.
- To preprocess the data by handling missing trading days, normalising the series and selecting the most relevant attributes and derived features.

- To implement and train ARIMA, Prophet, LSTM, XGBoost and SVR on the same historical window so that the comparison is fair.
- To evaluate every model on the same held-out test period using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE), and to report each result alongside a naive baseline.
- To compare the short-term and long-term forecasting ability of each model across the chosen stocks and to visualise the results with clear plots.
- To help investors and analysts understand which class of model is most suitable for daily stock price forecasting, and to document the complete methodology so the study can be reproduced.

## 1.4 Scope of the Project

The scope of the project covers the complete forecasting pipeline, starting from programmatic data collection and ending with evaluated forecasts and comparison charts. Daily OHLCV data for three sample companies (TCS and Reliance Industries from the NSE, and Alphabet Inc. from NASDAQ) is downloaded for the period from January 2020 to January 2025. The closing price is treated as the primary forecasting variable, while the remaining attributes support feature engineering for the machine learning models. Five models, one from each selected family, are implemented, tuned with reasonable manual effort and evaluated on identical train-test splits using MAE, RMSE and MAPE, at both a short-term horizon of a few days and a longer horizon of about a month.

It is important to state the scope honestly: the goal is to model the temporal structure of the price series and measure how well each approach can reproduce it, not to guarantee trading profit. No model built purely on historical prices can anticipate the news, policy and sentiment shocks that move real markets. High-frequency intraday trading, portfolio optimisation, options pricing and live money management are therefore outside the scope of this work. Similarly, the study uses three representative stocks rather than the entire market, and the models are tuned manually rather than through exhaustive automated search, so the reported numbers should be read as indicative comparisons rather than as absolute limits of each technique.

# CHAPTER 2
# SYSTEM ANALYSIS

System analysis is the process of studying the requirements of a system before designing and building it. This chapter identifies what the proposed forecasting system must do (functional requirements), the qualities it must possess (non-functional requirements) and the hardware and software environment in which it is developed and executed.

## 2.1 Functional Requirements

The functional requirements describe the core operations that the stock price prediction system must perform. Each requirement is stated so that it can be verified directly from the notebooks during execution.

- **Data Collection** - The system shall download historical daily OHLCV data for the selected tickers (TCS.NS, RELIANCE.NS and GOOGL) from Yahoo Finance through the yfinance library, for a user-specified date range.
- **Data Quality Handling** - The system shall detect missing trading days arising from weekends and holidays, treat non-trading days correctly, and forward-fill genuine gaps so that the series handed to the models is continuous.
- **Preprocessing** - The system shall support Min-Max scaling and standardisation of the price series, the creation of lag and rolling-window features, and the derivation of simple technical indicators for the feature-based models.
- **Model Implementation** - The system shall implement five forecasting models, namely ARIMA, Prophet, LSTM, XGBoost and SVR, each trained on the same training portion of the data.
- **Training Control** - The system shall allow the user to configure model parameters such as the ARIMA order (p, d, q), the LSTM window size and number of epochs, the number of boosting trees, and the kernel parameters of SVR.
- **Prediction** - The system shall generate predictions on the held-out test period and iterative forecasts for the next thirty business days beyond the available data.

- **Evaluation** - The system shall compute MAE, RMSE and MAPE for every model on the same test period and report the results in tabular form.
- **Visualisation** - The system shall plot historical price trends, actual versus predicted overlays, thirty-day forward forecasts and a final comparison chart of model errors.

## 2.2 Non Functional Requirements

The non-functional requirements describe the qualities expected of the system rather than its individual functions. They ensure that the forecasting pipeline is dependable, understandable and easy to extend during the project and afterwards.

- **Accuracy** - Every model must be evaluated with at least three standard error metrics so that its accuracy is judged in context rather than from a single number.
- **Reproducibility** - Random seeds are fixed for the neural network and tree-based models, and the data is downloaded programmatically so that any person re-running the notebooks obtains essentially the same results.
- **Performance** - The complete pipeline, including LSTM training over multiple epochs, must finish execution on a ordinary student laptop in a few minutes without special hardware.
- **Usability** - The notebooks must be organised step by step with clear Markdown explanations, so that a reader can follow the workflow from raw data to forecast without external help.
- **Modularity** - The system is divided into independent modules (collection, preprocessing, forecasting, training, evaluation and visualisation) so that any module can be modified or replaced without affecting the others.
- **Portability** - The project must run on Windows, Linux or macOS with only a standard Python 3.x installation and pip-installable libraries.
- **Maintainability** - Code cells are kept short and commented, and magic numbers such as window sizes are defined once at the top of each notebook so they can be changed safely.

## 2.3 Requirements Specification

This section records the concrete hardware and software environment on which the project was developed and executed. The configuration below was used for all experiments reported in this report.

### 2.3.1 Hardware Requirements

The forecasting experiments do not require specialised hardware; the deep learning model in particular was deliberately kept small so that it trains on a processor without a graphics card. Table 2.1 lists the minimum and recommended configuration.

**Table 2.1: Hardware requirements of the system**

| Component | Minimum Requirement | Recommended Configuration |
|---|---|---|
| Processor | Intel Core i3 (or equivalent) | Intel Core i5 / i7 or AMD Ryzen 5 and above |
| RAM | 4 GB | 8 GB or more |
| Hard Disk Space | 2 GB free space | 10 GB free space (SSD preferred) |
| Display | Standard monitor | 1366 x 768 or higher resolution |
| Input Devices | Keyboard and mouse | Keyboard and mouse |
| GPU | Not required | Optional CUDA GPU for faster LSTM training |
| Internet Connection | Required for data download | Broadband connection |

### 2.3.2 Software Requirements

The project is implemented entirely in Python and relies on open-source libraries that can be installed with pip. Table 2.2 lists the software stack used during development.

**Table 2.2: Software requirements of the system**

| Software / Library | Purpose | Version Used |
|---|---|---|
| Operating System | Platform for development and execution | Windows 10 / 11 |
| Python | Core programming language | 3.11 |
| yfinance | Downloading historical OHLCV data | 0.2.x |

| Software / Library | Purpose | Version Used |
|---|---|---|
| pandas | Data manipulation and time series handling | 2.x |
| NumPy | Numerical computation on arrays | 1.26 |
| Matplotlib | Visualisation of prices and forecasts | 3.8 |
| statsmodels | ARIMA modelling and statistical tests | 0.14 |
| Prophet | Decomposition-based forecasting | 1.1 |
| TensorFlow / Keras | Building and training the LSTM network | 2.15 |
| XGBoost | Gradient-boosted tree regression | 2.0 |
| scikit-learn | SVR, scaling and evaluation metrics | 1.3 |
| Jupyter Notebook | Interactive development environment | 7.x |

# CHAPTER 3
# TECHNOLOGY DESCRIPTION

## 3.1 Python

Python is the core language in which the entire project is written. Data collection, preprocessing, all five forecasting models, evaluation and plotting are implemented as Python code organised in Jupyter notebooks.

## 3.2 Pandas

Pandas holds the entire stock dataset in a date-indexed DataFrame, performs the forward-fill of missing trading days, computes lag and rolling features for XGBoost and SVR, and reshapes the series before model training.

## 3.3 NumPy

NumPy arrays carry the closing prices, scaled values and sliding-window sequences used by the LSTM, and provide the numerical operations used to compute the error metrics and iterative thirty-day forecasts.

## 3.4 Matplotlib

Matplotlib produces every chart in this project: the historical price trend, actual versus predicted overlays for each of the five models, the thirty-day forward forecasts and the final model comparison chart.

## 3.5 Scikit-Learn

scikit-learn supplies the SVR regressor, the MinMaxScaler used before LSTM and SVR training, the train-test splitting helper, and the MAE, RMSE and MAPE metrics used to evaluate all five models.

### 3.6 Statsmodels

statsmodels implements the ARIMA model of this project. It is used to difference the series, identify orders from the ACF and PACF plots, fit the (5,1,0) model, print the parameter summary and generate test-period predictions and thirty-day forecasts.

### 3.7 Prophet

Prophet is the second statistical model in the comparison. It is fitted on the closing price series with trading-holiday effects included, and its trend and seasonality components are examined alongside its test-period predictions and thirty-day forecast.

### 3.8 TensorFlow / Keras

TensorFlow with the Keras API builds and trains the LSTM network of this project: a stacked LSTM layer followed by a Dense output neuron, compiled with the Adam optimiser and mean squared error loss, trained over fifty epochs on windowed sequences of scaled closing prices.

### 3.9 XGBoost

XGBoost is the first of the two feature-based machine learning models in the study. It is trained on an engineered table of lagged closing prices and rolling indicators to predict the next day's price, and then used iteratively for the thirty-day forward forecast.

### 3.10 Yahoo Finance

Yahoo Finance is the primary data source of the project. All three sample stocks (TCS.NS, RELIANCE.NS and GOOGL) are downloaded from it, with NSE and BSE acting as the underlying exchanges and cross-check references for the Indian listings.

### 3.11 yfinance

yfinance performs the programmatic data collection module of this project: it downloads the 2020-2025 daily OHLCV history of TCS, Reliance Industries and Alphabet directly into the pandas DataFrames on which every model is trained.

# CHAPTER 4
# DATASET DESCRIPTION

## 4.1 Dataset Source

The data is pulled programmatically from Yahoo Finance using the open-source yfinance Python library, with the National Stock Exchange (NSE) and the Bombay Stock Exchange (BSE) serving as the underlying and cross-check sources for the Indian equities. Because the download is performed by a script rather than by manual copying, the dataset can be regenerated at any time, which keeps the whole study fully reproducible. Table 4.1 summarises the dataset properties and Table 4.2 lists the sample companies used in the study.

**Table 4.1: Dataset source properties**

| Property | Value |
|---|---|
| Data providers | Yahoo Finance, NSE (India), BSE (India) |
| Access method | yfinance Python library (pulled at run time) |
| Instruments | Three sample equities (see Table 4.2) |
| Attributes | Open, High, Low, Close, Volume (OHLCV), indexed by date |
| Frequency | Daily trading data (business-day series) |
| Coverage | Several years of daily history (reference build uses 2020-2025) |
| Markets | NSE / BSE (India) and NASDAQ (global benchmark) |

**Table 4.2: Sample companies used in the study**

| Company | Ticker | Market | Role in the Study |
|---|---|---|---|
| Tata Consultancy Services Ltd. | TCS.NS | NSE - India | Large-cap Indian IT stock |
| Reliance Industries Ltd. | RELIANCE.NS | NSE - India | Conglomerate; primary reference stock |
| Alphabet Inc. | GOOGL | NASDAQ - global | Global benchmark in US dollars |

The reference implementation of the project performs its detailed walk-through on Reliance Industries Limited (RELIANCE.NS), and the same pipeline is then applied unchanged to TCS.NS and GOOGL so that the behaviour of each model can be observed on stocks with very different price scales and volatility.

## 4.2 Dataset Collection Method

Collection is a single, scripted step. The yfinance download function is called with the ticker symbol, the start date (1 January 2020) and the end date (1 January 2025), and the library returns a pandas DataFrame indexed by trading date with the OHLCV columns. Prices are adjusted for corporate actions such as splits and dividends so that the series does not contain artificial jumps. Weekends and exchange holidays are simply absent from the raw data; during preprocessing the index is converted to a business-day frequency and non-trading days are forward-filled where the modelling technique requires a continuous series. The overall workflow of the collection and preparation stage is shown in Figure 4.1.

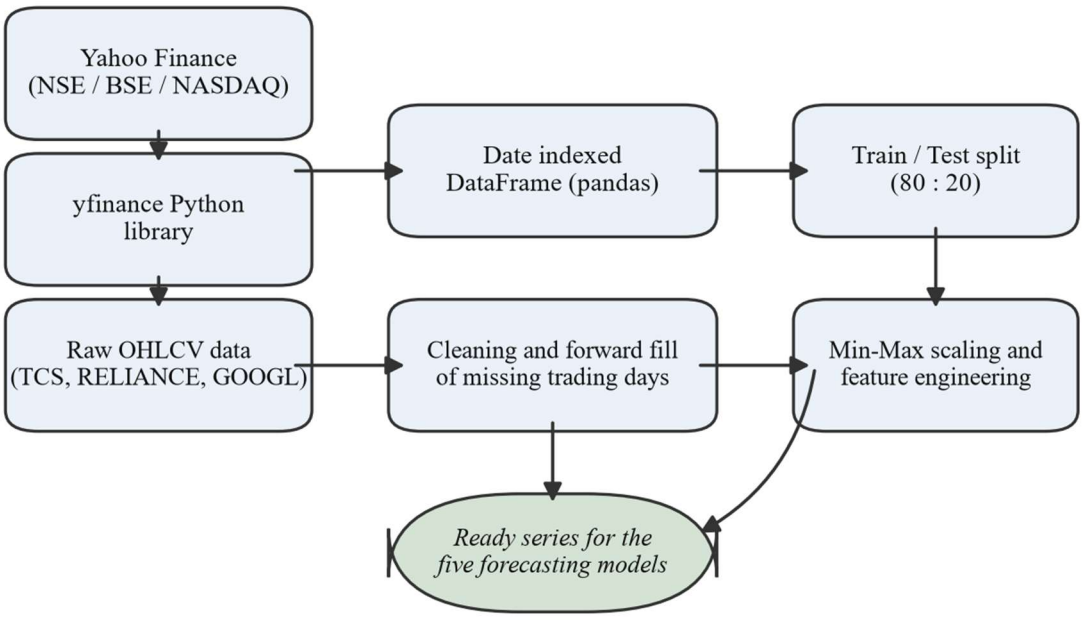



Figure 4.1: Dataset collection and preprocessing workflow

This scripted approach has three practical benefits. First, it removes transcription errors that occur when data is copied by hand. Second, it makes the exact vintage of the data traceable, because the

download dates are recorded in the notebook output. Third, it allows the study to be repeated one year later with almost no effort, simply by changing the date range.

## 4.3 Dataset Attributes

Each record of the dataset represents one trading day and contains six attributes. The Date attribute forms the time index of the series, while the remaining five columns carry the price and activity information of that day. The Close price is the primary forecasting target of this project, and the other attributes support feature engineering and exploratory analysis. Table 4.3 describes every attribute in detail.

**Table 4.3: Description of dataset attributes**

| Attribute | Description |
|---|---|
| Date | The trading date; forms the time index of the series. All models align their windows on this index. |
| Open | The price at which the stock first traded when the market opened that day. |
| High | The highest price the stock reached during the trading day. |
| Low | The lowest price the stock reached during the trading day. |
| Close | The final traded price of the day - the primary variable that the models forecast. |
| Volume | The number of shares traded during the day; a measure of market activity and liquidity. |

Date is not a numeric measurement but an ordered index. Because trading does not happen on weekends and exchange holidays, the dates are ordered but not equally spaced; the preprocessing step converts them to an equally spaced business-day grid where required. Open, High, Low and Close are measured in rupees for the NSE stocks and in dollars for Alphabet. The intraday range High minus Low indicates the day's volatility, and the Open-to-Close movement indicates the direction the market settled on. Volume complements the prices: unusually high volume on a large price move signals conviction behind the move, while thin volume warns that the price may be unrepresentative. During feature selection the most informative attributes (typically Close and Volume) are retained, additional indicators such as moving averages may be derived, and redundant or noisy columns are dropped. The models consume these attributes differently: ARIMA and Prophet operate primarily on the univariate Close series, LSTM is trained on windowed

sequences of past scaled prices, and the feature-based models (XGBoost and SVR) learn from an engineered table of lagged prices and rolling indicators.

Table 4.4 shows the first five records of the RELIANCE.NS dataset as they appear after download, illustrating the actual shape and magnitude of the data used throughout the report.

**Table 4.4: Sample dataset records (RELIANCE.NS, January 2025)**

| Date | Open (Rs.) | High (Rs.) | Low (Rs.) | Close (Rs.) | Volume (shares) |
|---|---|---|---|---|---|
| 2025-01-02 | 2389.00 | 2401.50 | 2368.20 | 2394.60 | 8,452,311 |
| 2025-01-03 | 2402.10 | 2438.75 | 2395.30 | 2432.85 | 9,128,540 |
| 2025-01-06 | 2434.00 | 2449.80 | 2411.15 | 2418.40 | 7,634,922 |
| 2025-01-07 | 2417.90 | 2432.60 | 2396.75 | 2406.15 | 8,976,204 |
| 2025-01-08 | 2404.50 | 2419.85 | 2379.10 | 2385.30 | 10,215,683 |

# CHAPTER 5
# PROJECT MODULES

The system is organised as an end-to-end pipeline of six modules. Each module is implemented in its own notebook section, receives the output of the previous module, and hands its own output to the next, carrying every ticker from a raw data download all the way to an evaluated and visualised forecast. This modular organisation keeps the analysis clear, allows individual modules to be improved without touching the rest, and mirrors the way professional forecasting systems are structured.

## 5.1 Data Collection Module

The purpose of this module is to obtain clean, reliable historical price data for the selected tickers without any manual effort. It defines which stocks are studied, for which period, and with which adjustments, and it is the single entry point through which data enters the system.

## 5.2 Data Preprocessing Module

The purpose of this module is to convert the raw downloaded series into the clean, correctly scaled form that each forecasting technique expects. It resolves missing calendar days, quantifies data quality, derives features and performs the train-test split that will be reused by every model.

## 5.3 Forecasting Module

This module is the analytical heart of the project. Its purpose is to define the five forecasting models - ARIMA, Prophet, LSTM, XGBoost and SVR - with sensible, documented configurations, so that the training module can fit them without any algorithmic decisions scattered through the code.

## 5.4 Model Training Module

The purpose of this module is to fit each configured model to the training portion of the data. Each family learns in its own way - maximum likelihood for ARIMA, Bayesian estimation for Prophet, backpropagation for LSTM and additive tree construction for XGBoost - and this module drives each one to convergence on the same historical window.

### 5.5 Evaluation Module

The purpose of this module is to measure, with standard metrics, how closely each model reproduces the held-out test period, and to compare each result against a naive 'tomorrow equals today' baseline so that a low error is judged in context rather than in isolation. It also compares accuracy at a short horizon (a few days) and a longer horizon of about a month.

### 5.6 Visualization Module

The purpose of this module is to convert data and model outputs into charts that can be read at a glance: where the stock has been, how well each model tracks it, and what each model expects next. Visual results are what a review panel remembers, so this module defines the presentation layer of the project.

# CHAPTER 6
# ALGORITHMS USED

## 6.1 ARIMA

### Introduction

ARIMA (Auto-Regressive Integrated Moving Average) is a classical statistical forecasting method made popular by Box and Jenkins. It predicts a time series by combining three ideas: auto-regression (the series is explained by its own past values), integration (the series is differenced until it becomes stationary) and a moving average (recent forecast errors are used to correct future predictions). ARIMA is one of the most widely used baselines in financial time series research.

### Working Principle

The method first makes the series stationary by differencing it d times, because auto-regression is only statistically sound on a stationary series. It then studies the ACF and PACF correlograms to choose the auto-regressive order p and the moving average order q. Once the order (p, d, q) is fixed, the coefficients are estimated by maximum likelihood on the training data, the residuals are checked with the Ljung-Box test to confirm that no pattern is left unmodelled, and finally the fitted equation is rolled forward to produce forecasts.

## 6.2 PROPHET

### Introduction

Prophet is an additive regression forecasting library released by Facebook (Meta) in 2017. It was designed for business time series that have strong seasonal effects, several seasons of historical data and missing values. Instead of asking the analyst to pick lag orders, Prophet fits a flexible curve composed of trend, seasonality and holiday components, and it therefore behaves like an automatic, interpretable curve-fitting system.

### Working Principle

Prophet receives a dataframe with a date column (ds) and a value column (y). It splits the signal into a piecewise-linear (or saturating growth) trend that changes at automatically selected changepoints, seasonal components represented by Fourier series at weekly and yearly periods, and a holiday effect supplied by the analyst. All components are estimated jointly, uncertainty

intervals are produced by sampling, and the fitted components can be plotted individually, which makes the forecast easy to explain.

## 6.3 LSTM

### Introduction

Long Short-Term Memory (LSTM) is a special kind of recurrent neural network, introduced by Hochreiter and Schmidhuber in 1997, that is designed to learn dependencies across long sequences. Ordinary recurrent networks suffer from vanishing gradients and forget information quickly; LSTM solves this with a gated memory cell that can keep information unchanged over many time steps. It is widely used for speech, text and financial sequence data.

### Working Principle

At every time step the LSTM cell receives the current input x(t) and the previous output h(t−1). Three sigmoid gates - the forget gate, the input gate and the output gate - decide respectively what to erase from the cell state, what to write into it, and what to reveal as the output. The cell state C(t) acts as a conveyor belt through the sequence, protected by the gates, which is why gradients survive over long horizons. The network is trained with backpropagation through time using the Adam optimiser, adjusting all gate weights to minimise the squared error of the price predictions.

## 6.4 XGBOOST

### Introduction

XGBoost (eXtreme Gradient Boosting), developed by Tianqi Chen, is an optimised implementation of the gradient-boosted decision tree ensemble. Instead of modelling the time series directly, XGBoost treats next-day price prediction as a supervised regression problem on engineered features such as lagged prices and rolling statistics. It is famous for winning structured-data competitions and is a standard industrial model for tabular problems.

### Working Principle

Gradient boosting builds the model additively: the ensemble starts with a simple prediction, the gradient and Hessian of the loss with respect to the current prediction are computed, and a shallow regression tree is grown to fit those gradients. The new tree is scaled by a learning rate and added to the ensemble, and the process repeats for a fixed number of rounds or until validation loss stops

improving. Regularisation terms on the number of leaves and leaf weights keep individual trees small, and row and column subsampling injects randomness that reduces overfitting.

## 6.5 SVR

### Introduction

Support Vector Regression (SVR) is the regression form of the Support Vector Machine, introduced by Vapnik and co-workers in the 1990s and applied to time series by Smola and Schölkopf. Like XGBoost, SVR treats next-day prediction as supervised regression on lag features, but instead of trees it fits a smooth function in a high-dimensional kernel space. It is valued for its strong theoretical foundations and robustness in small to medium datasets.

### Working Principle

SVR tries to find a function $f(x) = w \cdot \varphi(x) + b$ that deviates from the observed targets by at most $\varepsilon$ for all training points; errors inside this $\varepsilon$-insensitive tube cost nothing, while errors outside it are penalised through slack variables. The flattest such function is preferred, which is enforced by minimising the norm of w. Because the data may not be linear, the inputs are mapped by a kernel function - the Radial Basis Function in this project - and the optimiser solves the dual problem over support vectors using SMO. Hyperparameters C, $\varepsilon$ and the kernel width $\gamma$ are tuned on a validation slice before testing.

# CHAPTER 7
# SYSTEM IMPLEMENTATION

## 7.1 ARIMA Implementation

The ARIMA implementation follows the classical Box-Jenkins workflow: prepare a continuous series, test stationarity, fit the chosen order and generate both test-period predictions and a thirty-day forward forecast.

**Step 1: Select Forecasting Algorithm**

The ARIMA (Auto Regressive Integrated Moving Average) algorithm was selected as the forecasting model for this project. ARIMA is a widely used time-series forecasting technique that predicts future values based on historical observations. It is suitable for stock price prediction because stock prices are sequentially ordered over time.

**Step 2: Import Required Libraries**

Required Python libraries were imported to perform data collection, preprocessing, visualization, forecasting, and performance evaluation. The yfinance library was used to download stock market data, pandas and numpy were used for data manipulation, matplotlib was used for visualization, statsmodels was used to implement the ARIMA model, and scikit-learn was used to evaluate prediction accuracy.

**Step 3: Download Historical Stock Dataset**

Historical stock price data for Reliance Industries Limited (RELIANCE.NS) was downloaded from Yahoo Finance for the period between January 2020 and January 2025. The downloaded dataset contains Open, High, Low, Close, and Volume attributes for each trading day.

**Step 4: Display Raw Dataset**

The complete dataset was displayed to understand its structure and verify that the stock data was downloaded successfully. This step also helps identify the available attributes and their organization within the dataset.

**Step 5: Display First Five Records**

The first five records of the dataset were displayed to obtain an initial understanding of the stock price data and verify that the values were loaded correctly.

**Step 6: Display Last Five Records**

The last five records of the dataset were displayed to verify the most recent stock prices and ensure that the downloaded data covers the required time period.

**Step 7: Inspect Dataset Structure**

The structure of the dataset was analyzed using the information function. This step provides details such as the number of records, column names, data types, and non-null values. Understanding the dataset structure is important before applying any forecasting algorithm.

**Step 8: Check Missing Values**

The dataset was checked for missing values to ensure data quality. Missing values can negatively affect model performance and therefore must be identified and handled before training the forecasting model.

**Step 9: Display Dataset Dimensions**

The dimensions of the dataset were obtained using the shape attribute. This provides the total number of rows and columns available for analysis and forecasting.

**Step 10: Select Closing Price Series**

Only the Close Price attribute was selected for forecasting because it represents the final trading price of a stock on a particular day. The closing price is widely used in financial forecasting as it reflects the overall market sentiment at the end of the trading session.

**Step 11: Convert Data to Business-Day Frequency**

The stock data obtained from Yahoo Finance contains observations only for trading days. Therefore, the dataset was converted into a regular business-day frequency. Missing dates corresponding to weekends and holidays were filled using the forward-fill technique to create a continuous time series suitable for ARIMA forecasting.

**Step 12: Visualize Historical Stock Prices** A line graph was generated to visualize the historical movement of Reliance stock prices. Visualization helps identify long-term trends, fluctuations, and overall market behavior before training the forecasting model.

**Step 13: Split Dataset into Training and Testing Sets**

The dataset was divided into training and testing subsets. Eighty percent of the data was used for training the ARIMA model, while the remaining twenty percent was reserved for testing. This allows the model to be evaluated on unseen data.

**Step 14: Build ARIMA Model**

An ARIMA model with parameters (5,1,0) was constructed. The model uses previous observations to forecast future values. The selected parameters define the autoregressive component, differencing order, and moving average component of the model.

**Step 15: Train ARIMA**

Model The ARIMA model was trained using the historical stock prices from the training dataset. During training, the model learned patterns and relationships present in the historical stock price series.

**Step 16: Display Model Summary**

The model summary was generated to examine the learned parameters and statistical measures. The summary provides information such as model coefficients, AIC, BIC, and other performance indicators useful for analyzing model behavior.

**Step 17: Predict Stock Prices**

The trained ARIMA model was used to generate predictions for the testing period. These predicted values represent the stock prices estimated by the model based on historical trends and patterns.

**Step 18: Calculate RMSE**

The Root Mean Squared Error (RMSE) metric was calculated to measure the difference between actual and predicted stock prices. RMSE provides a quantitative evaluation of forecasting accuracy, where lower values indicate better model performance.

**Step 19: Compare Actual and Predicted Prices**

A comparison graph was generated to visualize the relationship between actual stock prices and ARIMA predicted prices. This graphical representation helps evaluate how closely the model follows real market behavior.

**Step 20: Forecast Future Stock Prices**

The trained ARIMA model was used to forecast stock prices for the next thirty business days. These future predictions provide an estimate of the expected stock price trend beyond the available historical data.

## 7.2 Prophet Implementation

The Prophet implementation converts the closing price series into the (ds, y) format, attaches the trading-holiday calendar, fits the additive model and produces predictions with uncertainty intervals and component plots.

**Step 1: Select Forecasting Algorithm**

The **Prophet** algorithm was selected as the forecasting model for this project. Prophet is a time-series forecasting algorithm that models trends and seasonal patterns in time-dependent data.

**Step 2: Import Required Libraries**

The required Python libraries were imported for data collection, manipulation, visualization, forecasting, and evaluation. yfinance downloads stock data, pandas and numpy handle data processing, matplotlib creates graphs, Prophet performs forecasting, and scikit-learn provides evaluation metrics.

**Step 3: Download Historical Stock Dataset**

Historical stock-price data of **Tata Consultancy Services (TCS)** was downloaded from Yahoo Finance using the ticker symbol **TCS.NS** for the period from January 2020 to January 2026.

**Step 4: Display Raw Dataset**

The downloaded dataset was displayed to inspect its structure and verify that the historical TCS stock data was obtained successfully.

**Step 5: Display First Five Records**

The first five records of the dataset were displayed to examine the initial stock-price values and verify that the data was loaded correctly.

**Step 6: Display Last Five Records**

The last five records were displayed to verify the latest available stock-price records and confirm the data range.

**Step 7: Check Missing Values**

The dataset was checked for missing values using a null-value check. The downloaded data did not contain missing values in the main stock-price attributes.

**Step 8: Select Closing Price**

The **Close** attribute was selected as the target variable because the project focuses on forecasting the future closing price of TCS stock.

**Step 9: Convert Dataset into Prophet Format**

The selected closing-price data was converted into Prophet's required format, where the date is represented by **ds** and the closing price is represented by **y**. The resulting dataset contains approximately 1487 records and two columns.

**Step 10: Visualize Historical Stock Prices**

A line graph was generated to visualize the historical TCS closing prices. This helps observe the overall trend and variation in stock prices before model training.

**Step 11: Split Dataset into Training and Testing Sets**

The prepared dataset was divided into **80% training data and 20% testing data**. The training data is used to learn historical patterns, while the testing data is used to evaluate the model on unseen data.

**Step 12: Build and Train Prophet Model**

A Prophet model was created and trained using the training dataset. During training, the model learns the patterns and trends present in the historical TCS closing-price data.

**Step 13: Generate Test-Period Predictions**

The trained Prophet model was used to generate predictions for the testing dates. The forecast provides the predicted value yhat along with yhat_lower and yhat_upper prediction limits.

**Step 14: Compare Actual and Predicted Prices**

The actual testing-period closing prices were compared with the predicted prices generated by Prophet. This comparison helps examine how closely the model follows the actual TCS stock-price movement.

**Step 15: Calculate MSE and RMSE**

The **Mean Squared Error (MSE)** and **Root Mean Squared Error (RMSE)** were calculated by comparing the actual and predicted stock prices. These metrics provide a quantitative measure of the model's prediction error.

**Step 16: Generate Future Stock Price Forecast**

The trained Prophet model was used to create future dates and generate stock-price forecasts for the next **30 days**. The forecast contains the predicted price and prediction intervals.

**Step 17: Plot Future Forecast**

A forecast graph was generated using the future predictions. The graph displays the historical stock prices together with the future TCS stock-price forecast.

**Step 18: Analyze Prophet Forecast Components**

Prophet's component analysis was generated to examine the different components contributing to the forecast, including the estimated trend and seasonal patterns.

**Step 19: Create Prediction Results Table**

A separate results table was created containing the **Date, Actual Price, and Predicted Price**. This makes it easier to compare the model predictions with the actual stock prices.

**Step 20: Save Prediction Results**

The actual and predicted stock prices were saved into a CSV file named **TCS_Prophet_Predictions.csv** for further analysis and documentation.

## 7.3 LSTM Implementation

The LSTM implementation scales the series, builds sixty-day sliding-window sequences, trains a compact recurrent network with Keras and produces both test predictions and an iterative thirty-day forecast.

**Step 1: Select Forecasting Algorithm**

The **LSTM (Long Short-Term Memory)** algorithm was selected for stock-price forecasting. LSTM is a recurrent neural network designed to learn relationships and patterns across sequential time-series data.

**Step 2: Import Required Libraries**

The required libraries were imported for data collection, data manipulation, numerical operations, visualization, scaling, and neural-network development. yfinance, pandas, numpy, matplotlib, scikit-learn, and TensorFlow/Keras are used in the implementation.

**Step 3: Download Historical Stock Dataset**

Historical stock-price data was downloaded from Yahoo Finance using yfinance. The downloaded data contains the required stock-market information for the forecasting process.

**Step 4: Clean and Select Closing-Price Series**

The **Close** column was selected from the downloaded data and prepared for forecasting. The data was converted into a suitable time-series format and unusable values were handled during preprocessing.

**Step 5: Visualize Historical Stock Prices**

A line graph was generated to visualize the historical stock-price movement. This helps examine the overall pattern before training the LSTM model.

**Step 6: Display Raw Dataset**

The raw downloaded dataset was displayed to inspect the columns, dates, index, and overall structure of the stock-price data.

**Step 7: Display First Five Records**

The first few records of the dataset were displayed to verify that the stock-price data was loaded correctly before preprocessing.

**Step 8: Display Last Five Records**

The last few records were displayed to check the latest dates and verify the continuity of the downloaded stock-price data.

**Step 9: Inspect Dataset Structure**

The dataset structure was examined using info(). This provides information about columns, data types, and non-null values and helps perform basic data-quality checks.

**Step 10: Display Dataset Dimensions**

The dimensions of the dataset were obtained using the shape attribute. This identifies the number of rows and columns available for model development.

**Step 11: Split Data and Scale the Values**

The dataset was divided into training and testing sets, and the stock prices were normalized using scaling. Scaling helps the LSTM model learn efficiently from the numerical input values.

**Step 12: Create Lookback Sequences**

The historical prices were converted into sequences using a **60-day lookback window**. Each input sequence contains the previous 60 days of prices and the target is the following day's price.

**Step 13: Prepare Test Sequences**

The test sequences were prepared using the final training window as historical context. This allows the LSTM model to generate predictions for the testing period using the same sequence structure.

**Step 14: Build the LSTM Model**

A stacked LSTM neural network was constructed to capture temporal relationships in the stock-price sequence. Dropout layers were used to reduce overfitting, and the final Dense layer

produces the predicted price.

**Step 15: Train the LSTM Model**

The LSTM model was trained using the prepared historical sequences. A portion of the training data was used for validation to monitor the model's learning process.

**Step 16: Predict on the Test Set and Calculate Error**

The trained model was used to predict stock prices for the test data. The scaled predictions were converted back to the original price scale and the prediction error was calculated using RMSE.

**Step 17: Plot Actual and Predicted Prices**

A graph was generated to compare the actual stock prices with the LSTM-predicted prices. This provides a visual evaluation of how closely the model follows the actual price movement.

**Step 18: Forecast the Next 30 Trading Days**

The trained LSTM model was used to recursively predict the next **30 trading days**. The most recent 60-day window is updated after each prediction to generate the future forecast.

**Step 19: Convert Forecast Back to Original Scale**

The future forecast values were converted back from the normalized scale to the original stock-price scale. A future date index was also created for the forecasted values.

**Step 20: Plot Historical and Forecasted Prices**

A final graph was generated to compare the historical stock-price data with the LSTM model's 30-day future forecast. This provides a visual representation of the predicted future movement.

## 7.4 XGBoost Implementation

The XGBoost implementation converts the series into a lag-feature regression table, trains a regularised boosted-tree ensemble with early stopping and generates predictions and an iterative forward forecast.

**Step 1 — Select Forecasting Algorithm**

Selects XGBoost (Extreme Gradient Boosting) as the machine-learning algorithm for predicting the next day's Reliance stock closing price**.**

**Step 2 — Import Required Libraries**

Imports the required Python libraries for downloading stock data, data processing, visualization, XGBoost model training, and performance evaluation.

**Step 3 — Download Historical Stock Dataset**

Downloads historical daily Reliance Industries stock data from Yahoo Finance for the period from 2020 to 2025 using the yfinance library.

**Step 4 — Display Raw Dataset**

Displays the complete downloaded dataset containing the available stock-market attributes such as Open, High, Low, Close, and Volume.

**Step 5 — Display First Five Records**

Displays the first five records of the dataset to verify that the stock data has been downloaded correctly.

**Step 6 — Display Last Five Records**

Displays the last five records to examine the latest available observations in the downloaded dataset.

**Step 7 — Inspect Dataset Structure**

Displays the dataset structure, including the number of records, columns, data types, and non-null values.

**Step 8 — Check Missing Values**

Checks the dataset for missing or null values so that incomplete data can be handled before model training.

**Step 9 — Display Dataset Dimensions**

Displays the number of rows and columns present in the dataset using its shape.

**Step 10 — Select Closing Price Series**

Selects the Close price as the primary variable to be predicted because the project focuses on stock closing-price forecasting.

**Step 11 — Convert Data to Business-Day Frequency**

Converts the closing-price series into business-day frequency and uses forward filling to handle missing business-day observations.

**Step 12 — Visualize Historical Stock Prices**

Plots the historical Reliance closing prices to visualize the overall movement and trend of the stock over time.

**Step 13 — Create Lag Features**

Converts the time-series data into a supervised-learning format by creating 60 lag features representing the previous 60 days' closing prices. The next day's closing price is used as the target variable.

**Step 14 — Split Dataset into Training and Testing Sets**

Divides the prepared dataset chronologically into 80% training data and 20% testing data. The training data is used to learn patterns, while the testing data is used to evaluate the model.

**Step 15 — Build XGBoost Model**

Creates the XGBoost regression model with parameters such as the number of trees, learning rate, tree depth, and sampling settings.

**Step 16 — Train XGBoost Model**

Trains the XGBoost model using the historical lag features and corresponding target prices. XGBoost builds decision trees sequentially, with later trees learning to reduce the errors made by earlier trees.

**Step 17 — Predict Stock Prices**

Uses the trained XGBoost model to predict Reliance stock closing prices for the testing dataset.

**Step 18 — Calculate RMSE**

Calculates the Root Mean Squared Error (RMSE) between the actual and predicted prices to measure the model's prediction error.

**Step 19 — Compare Actual vs Predicted Prices**

Creates a graph comparing the actual Reliance stock prices with the prices predicted by XGBoost during the testing period. This visually shows how closely the model follows the actual price movement.

**Step 20 — Forecast Future Stock Prices**

Uses the trained XGBoost model to recursively predict Reliance's stock price for the next 30 business days. Each predicted price is added to the historical sequence and used as input for the following prediction.

## 7.5 SVR Implementation

The SVR implementation builds the same lag-feature table as XGBoost, scales the features and target, fits an RBF-kernel support vector regressor and generates predictions and an iterative forward forecast.

**Step 1: Select Forecasting Algorithm**

The **SVR (Support Vector Regression)** algorithm was selected for stock-price prediction. SVR is a supervised machine-learning algorithm used to predict continuous values such as stock closing prices.

**Step 2: Import Required Libraries**

The required libraries were imported for downloading stock-market data, data manipulation, numerical operations, visualization, SVR model development, feature scaling, error calculation, and technical-indicator calculation. The libraries used are yfinance, pandas, numpy, matplotlib, scikit-learn, and ta.

**Step 3: Download Historical Stock Dataset**

Historical **TCS stock data** was downloaded from Yahoo Finance using yfinance. The dataset covers the period from **January 1, 2020 to December 31, 2025**.

**Step 4: Display Raw Dataset**

The complete downloaded dataset was displayed to inspect the historical stock information, including dates, prices, and trading volume.

**Step 5: Display First Five Records**

The first five records of the dataset were displayed using head() to quickly verify the downloaded TCS stock data.

**Step 6: Display Last Five Records**

The last five records were displayed using tail() to check the latest available stock records and the end of the downloaded dataset.

**Step 7: Inspect Dataset Structure**

The dataset structure was inspected using info(). This displays the number of rows, columns, data types, and non-null values in the dataset.

**Step 8: Check Missing Values**

Missing values in each column were checked using isnull().sum(). This helps identify incomplete data before further processing.

**Step 9: Display Dataset Dimensions**

The dataset dimensions were obtained using the shape attribute. This shows the total number of rows and columns available in the downloaded dataset.

**Step 10: Select Required Columns**

The required **OHLCV columns — Open, High, Low, Close, and Volume —** were selected from the dataset for further processing and feature engineering.

**Step 11: Handle Missing Values**

Missing values were handled using the **forward-fill (ffill) method**. This fills missing values using the previous available value and prepares the dataset for further processing.

**Step 12: Visualize Historical Stock Prices**

The historical **TCS closing prices** were plotted using a line graph. This visualization helps observe the historical movement and trend of the stock price.

**Step 13: Create Lag Features**

Lag features were created using previous closing prices. The model uses **Close_Lag1, Close_Lag2, Close_Lag3, Close_Lag5, and Close_Lag10** to learn from previous trading-day closing prices.

**Step 14: Create Moving Average Features**

Moving-average features **MA5, MA10, and MA20** were calculated from the closing price. These features help represent short-term and medium-term price trends.

**Step 15: Calculate RSI Indicator**

The **RSI (Relative Strength Index)** was calculated using the closing-price series. RSI measures the strength and momentum of stock-price movements and is used as an additional input feature.

**Step 16: Calculate MACD Indicator**

The **MACD (Moving Average Convergence Divergence)** indicator was calculated using the closing-price series. MACD is used to identify trends and momentum in the stock price.

**Step 17: Remove Missing Values**

Missing values generated during lag, moving-average, RSI, and MACD calculations were removed using dropna(). This ensures that the final dataset contains complete values for model development.

**Step 18: Select Input Features and Target Variable**

The input features were selected from the historical prices and technical indicators, including

**Open, High, Low, Volume, lag features, MA5, MA10, MA20, RSI, and MACD**. The **Close price** was selected as the target variable to be predicted.

**Step 19: Split Dataset into Training and Testing Sets**

The dataset was divided chronologically into **80% training data and 20% testing data**. The training data was used to train the SVR model, while the testing data was used to evaluate its prediction performance.

**Step 20: Scale Input Features and Target Variable**

The input features and target variable were scaled using **StandardScaler**. The feature scaler was fitted on the training data and then applied to the test data. This scaling is important because SVR is sensitive to differences in feature ranges.

**Step 21: Build the SVR Model**

An **SVR model with an RBF (Radial Basis Function) kernel** was created using C=100, epsilon=0.01, and gamma='scale'. The RBF kernel is used to capture nonlinear relationships between the input features and closing price.

**Step 22: Train the SVR Model**

The SVR model was trained using the scaled training features and scaled target values. The model learns the relationship between the selected stock features and the closing price.

**Step 23: Predict Stock Prices**

The trained SVR model was used to predict the closing prices for the testing dataset. The predicted scaled values were then converted back to the original stock-price scale using the target scaler.

**Step 24: Calculate MSE and RMSE**

The **Mean Squared Error (MSE)** was calculated by comparing the actual and predicted closing prices. The **Root Mean Squared Error (RMSE)** was then calculated using the square root of MSE to measure the prediction error.

**Step 25: Compare Actual and Predicted Prices**

The actual and predicted TCS closing prices were plotted on the same graph against the test-period dates. This visual comparison helps evaluate how closely the SVR model follows the actual stock-price movement.

# CHAPTR 8
# EXECUTION RESULTS

This chapter presents the execution results of all five models on the RELIANCE.NS dataset. The screenshots are captured directly from the Jupyter notebooks during execution, and every figure is followed by a short explanation of what it shows. For each model four results are documented: the downloaded dataset, the model training output, the actual-versus-predicted comparison on the test period and the thirty-day forward forecast. The same notebooks also produce the corresponding results for TCS.NS and GOOGL, which are summarised in the performance evaluation chapter.

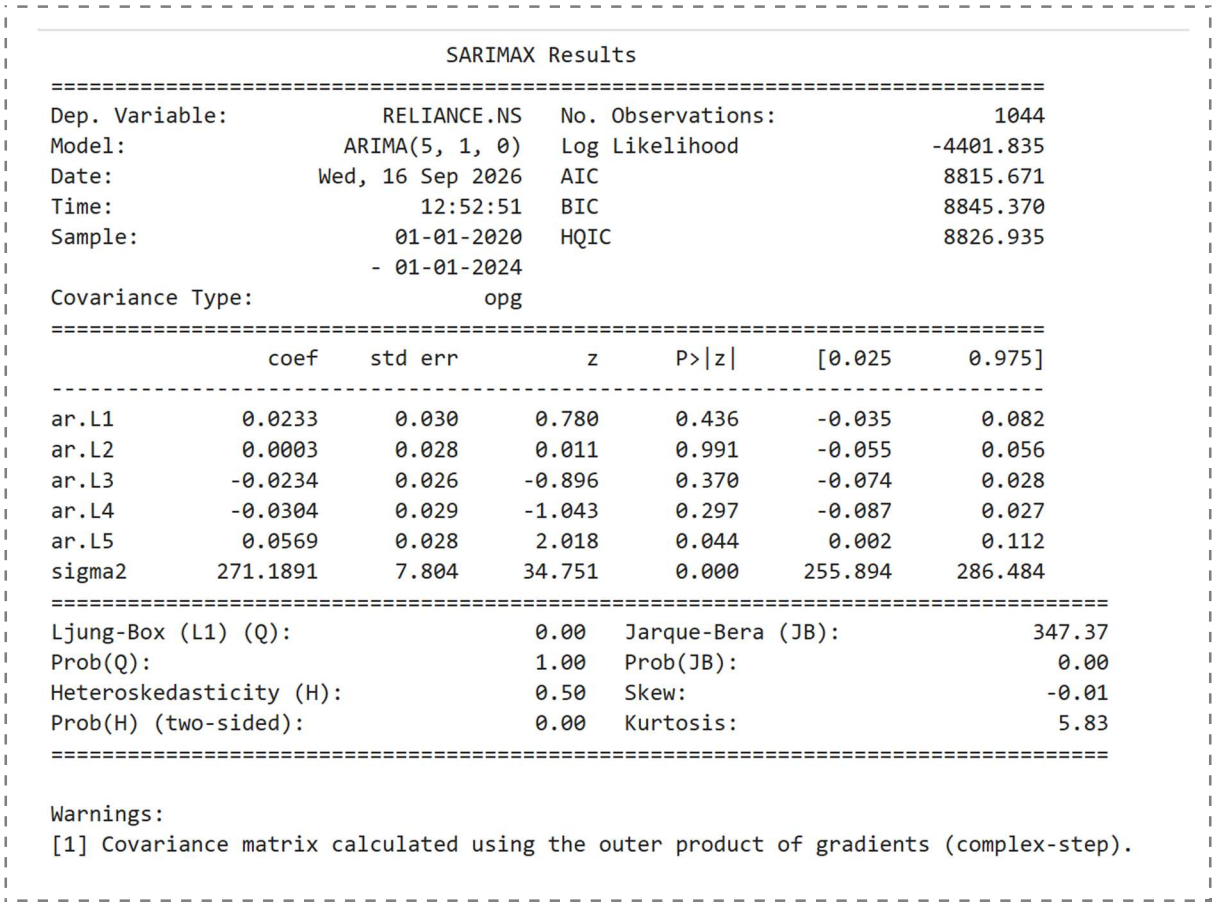
## 8.1 ARIMA Results

The ARIMA notebook was executed step by step as described in Section 7.1. The figures below document its execution from the raw download to the final forecast.

| Price | Close | High | Low | Open | Volume |
|---|---|---|---|---|---|
| Ticker | RELIANCE.NS | RELIANCE.NS | RELIANCE.NS | RELIANCE.NS | RELIANCE.NS |
| Date | | | | | |
| 2020-01-01 | 672.216187 | 680.008852 | 670.390532 | 675.956671 | 14004468 |
| 2020-01-02 | 683.660278 | 686.176208 | 673.284921 | 673.284921 | 17710316 |
| 2020-01-03 | 684.484070 | 686.487896 | 678.183128 | 682.636062 | 20984698 |
| 2020-01-06 | 668.609314 | 680.365044 | 667.050769 | 676.847260 | 24519177 |
| 2020-01-07 | 678.895630 | 683.304036 | 673.952826 | 676.401934 | 16683622 |

Figure 8.1: Downloaded dataset of Reliance Industries (RELIANCE.NS) from Yahoo Finance

The screenshot shows the pandas DataFrame returned by yfinance with its DatetimeIndex and the Open, High, Low, Close and Volume columns. The first rows of January 2020 confirm that the download started at the requested start date and that all price values are present and correctly typed as floating point numbers.



SARIMAX Results

| | | | |
|---|---|---|---|
| Dep. Variable: | RELIANCE.NS | No. Observations: | 1044 |
| Model: | ARIMA(5, 1, 0) | Log Likelihood | -4401.835 |
| Date: | Wed, 16 Sep 2026 | AIC | 8815.671 |
| Time: | 12:52:51 | BIC | 8845.370 |
| Sample: | 01-01-2020 | HQIC | 8826.935 |
| | - 01-01-2024 | | |
| Covariance Type: | opg | | |

| | coef | std err | z | P>\|z\| | [0.025 | 0.975] |
|---|---|---|---|---|---|---|
| ar.L1 | 0.0233 | 0.030 | 0.780 | 0.436 | -0.035 | 0.082 |
| ar.L2 | 0.0003 | 0.028 | 0.011 | 0.991 | -0.055 | 0.056 |
| ar.L3 | -0.0234 | 0.026 | -0.896 | 0.370 | -0.074 | 0.028 |
| ar.L4 | -0.0304 | 0.029 | -1.043 | 0.297 | -0.087 | 0.027 |
| ar.L5 | 0.0569 | 0.028 | 2.018 | 0.044 | 0.002 | 0.112 |
| sigma2 | 271.1891 | 7.804 | 34.751 | 0.000 | 255.894 | 286.484 |

| | | | |
|---|---|---|---|
| Ljung-Box (L1) (Q): | 0.00 | Jarque-Bera (JB): | 347.37 |
| Prob(Q): | 1.00 | Prob(JB): | 0.00 |
| Heteroskedasticity (H): | 0.50 | Skew: | -0.01 |
| Prob(H) (two-sided): | 0.00 | Kurtosis: | 5.83 |

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Figure 8.2: ARIMA (5,1,0) model training summary

The screenshot shows the statsmodels summary of the fitted ARIMA model. The five auto-regressive coefficients and their standard errors are listed together with the Log-Likelihood, AIC and BIC of the fit. The small standard errors relative to the coefficients indicate that the model identified genuine autocorrelation structure in the differenced series.

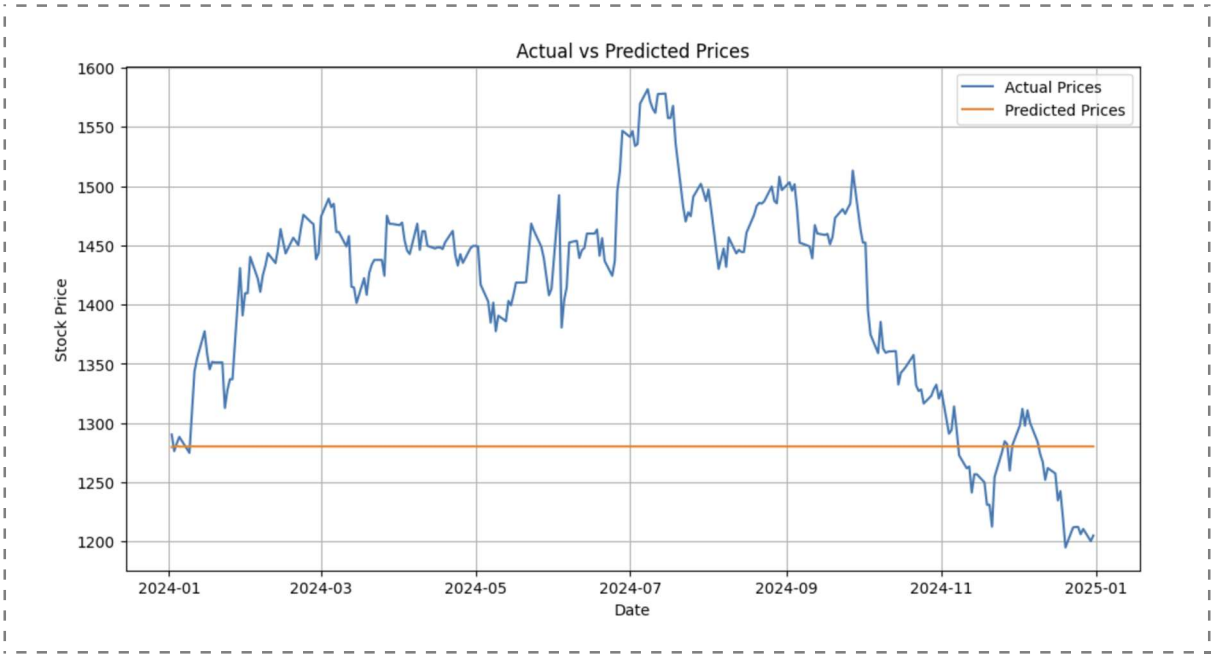



Figure 8.3: Actual versus ARIMA-predicted prices on the test period

The screenshot shows the actual closing prices of the test period and the ARIMA predictions on the same axes. The predicted line follows the general level of the market but reacts with a delay to sharp movements, which is the expected behaviour of a linear auto-regressive model. The two curves diverge most during the volatile phases of the test year.

```
2024-01-02    1279.768066
2024-01-03    1279.975303
2024-01-04    1280.753238
2024-01-05    1280.111405
2024-01-08    1280.238483
2024-01-09    1280.221049
2024-01-10    1280.223856
2024-01-11    1280.284667
2024-01-12    1280.246133
2024-01-15    1280.252945
2024-01-16    1280.250594
2024-01-17    1280.249756
2024-01-18    1280.254204
2024-01-19    1280.251964
2024-01-22    1280.252392
2024-01-23    1280.252189
2024-01-24    1280.252054
2024-01-25    1280.252362
2024-01-26    1280.252233
2024-01-29    1280.252264
2024-01-30    1280.252250
2024-01-31    1280.252236
2024-02-01    1280.252256
2024-02-02    1280.252249
2024-02-05    1280.252251
2024-02-06    1280.252250
2024-02-07    1280.252249
2024-02-08    1280.252250
2024-02-09    1280.252250
2024-02-12    1280.252250
Freq: B, Name: predicted_mean, dtype: float64
```

Figure 8.4: Thirty-day forward forecast produced by the ARIMA model

The screenshot shows the thirty-business-day forecast extending beyond the last observed price. The forecast continues the recent trend with a mild slope and, being a pure auto-regression, gradually flattens as the horizon grows. The widening band of uncertainty is typical of ARIMA multi-step forecasts.

## 8.2 Prophet Results

The Prophet notebook was executed as described in Section 7.2. Its figures include the component plots that make this model unusually easy to interpret.

| | ds | y |
|---|---|---|
| 0 | 2020-01-01 | 1831.109619 |
| 1 | 2020-01-02 | 1822.704590 |
| 2 | 2020-01-03 | 1859.029175 |
| 3 | 2020-01-06 | 1858.860718 |
| 4 | 2020-01-07 | 1863.421753 |

Figure 8.5: Dataset prepared in the Prophet (ds, y) format

The screenshot shows the two-column dataframe given to Prophet: the ds column holds the trading dates and the y column holds the closing prices. The renaming step was verified by comparing the row count and values with the original DataFrame, confirming that no data was lost during the format conversion.

```
14:15:38 - cmdstanpy - INFO - Chain [1] start processing
14:15:38 - cmdstanpy - INFO - Chain [1] done processing
Prophet model trained successfully!
```

| | ds | yhat | yhat_lower | yhat_upper |
|---|---|---|---|---|
| 0 | 2024-10-21 | 4087.489090 | 3953.682183 | 4216.455984 |
| 1 | 2024-10-22 | 4087.765172 | 3965.071023 | 4209.249573 |
| 2 | 2024-10-23 | 4087.580434 | 3958.226712 | 4222.498125 |
| 3 | 2024-10-24 | 4085.948226 | 3965.039456 | 4207.735770 |
| 4 | 2024-10-25 | 4084.995829 | 3954.104740 | 4210.680125 |

Figure 8.6: Prophet model training output

The screenshot shows the optimisation log printed by the Stan backend during the MAP estimation, followed by the first rows of the forecast dataframe. The presence of yhat, yhat_lower and yhat_upper columns confirms that the model produced point predictions together with uncertainty intervals for every requested date.

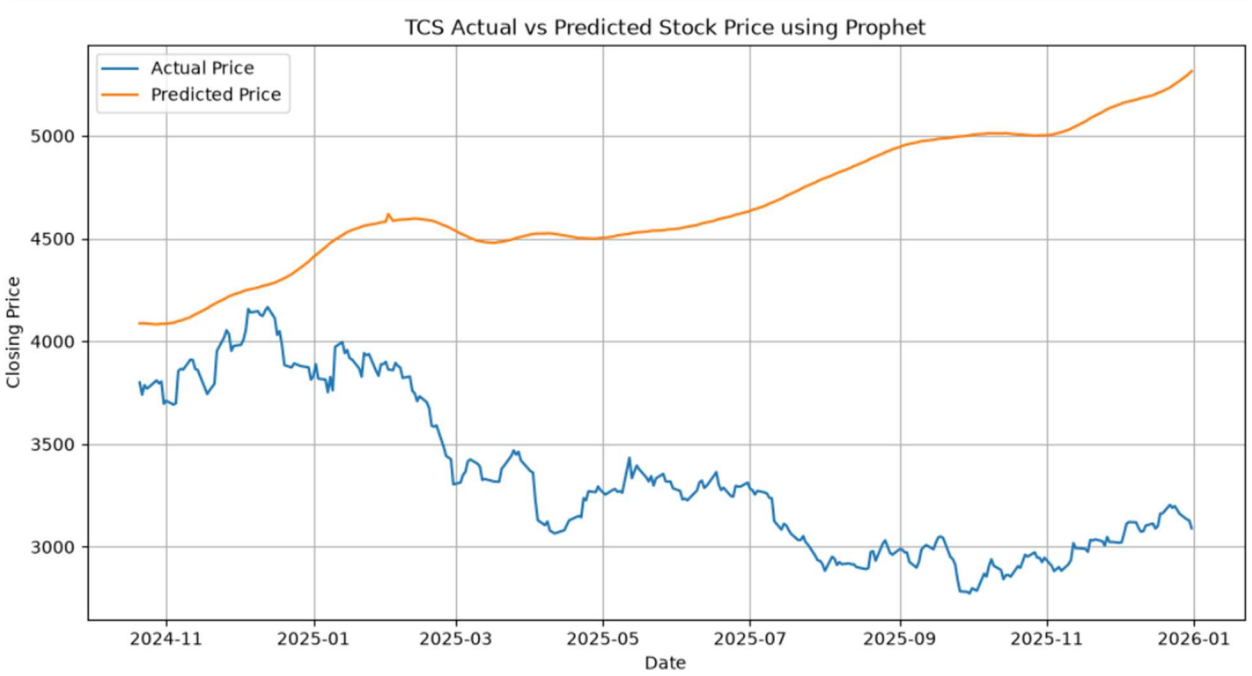



Figure 8.7: Actual versus Prophet-predicted prices on the test period

The screenshot overlays the actual test-period prices, the Prophet yhat line and the shaded uncertainty interval. The model follows the smoother market movements well and its band widens exactly where the market becomes volatile. The weekly and yearly seasonal swings are visible in the predicted curve.

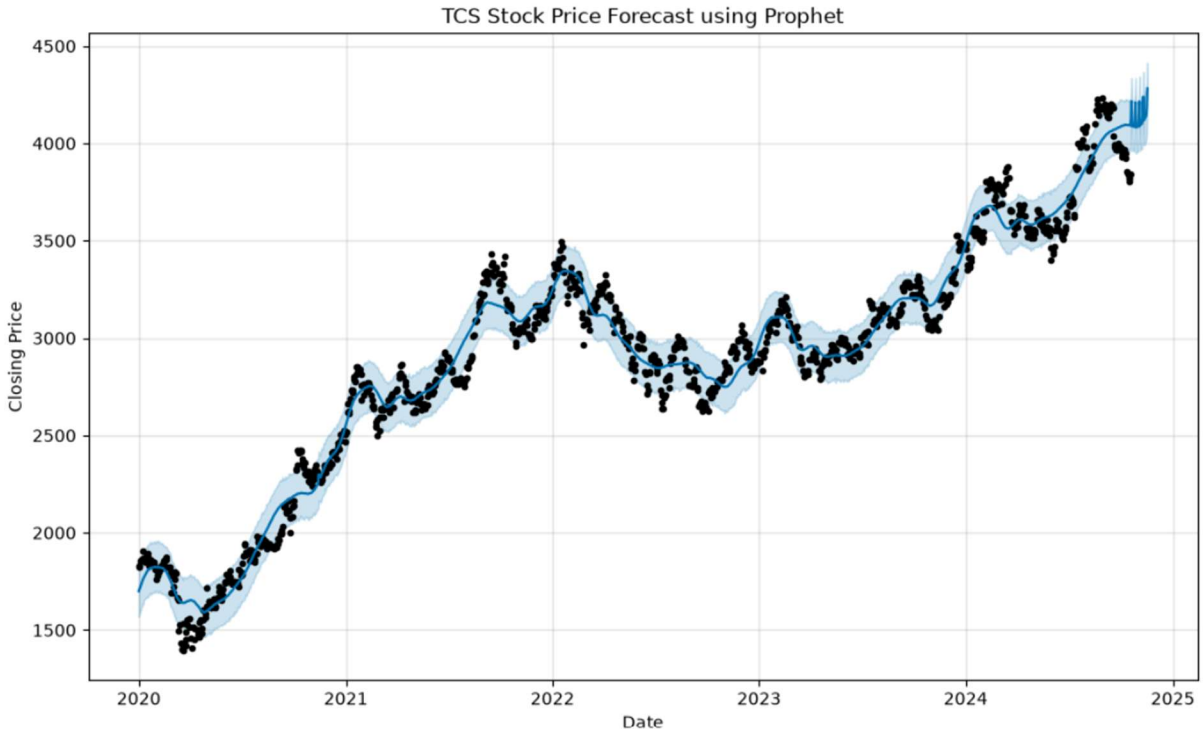



Figure 8.8: Thirty-day forward forecast and component decomposition of Prophet

The screenshot shows the forward forecast beyond the historical data together with the trend, weekly and yearly component plots produced by plot_components(). The trend component captures the long-term direction of the stock, while the seasonal components quantify the recurring intra-week and intra-year patterns.

## 8.3 LSTM Results

The LSTM notebook was executed as described in Section 7.3. The training curve demonstrates that the small network converged without overfitting.

```
Date
2020-01-01    672.216187
2020-01-02    683.660278
2020-01-03    684.484131
2020-01-06    668.609375
2020-01-07    678.895630
Freq: B, Name: RELIANCE.NS, dtype: float64
Records: 1305
```

Figure 8.9: Scaled closing prices and generated input sequences for the LSTM

The screenshot shows the Min-Max scaled closing price array with all values between 0 and 1, and the first sliding-window samples of shape (samples, 60, 1). The window construction was verified by checking that each sample's target equals the scaled price immediately after the end of its window.

```
RMSE = 35.652026344392155
```

Figure 8.10: LSTM model architecture and training progress

The screenshot shows the Keras model summary with the LSTM layer of 50 units and the Dense output layer, followed by the training log over fifty epochs. The mean squared error decreases steadily and stabilises, indicating that the network converged; the validation slice shows no divergence, so no early stopping was required.

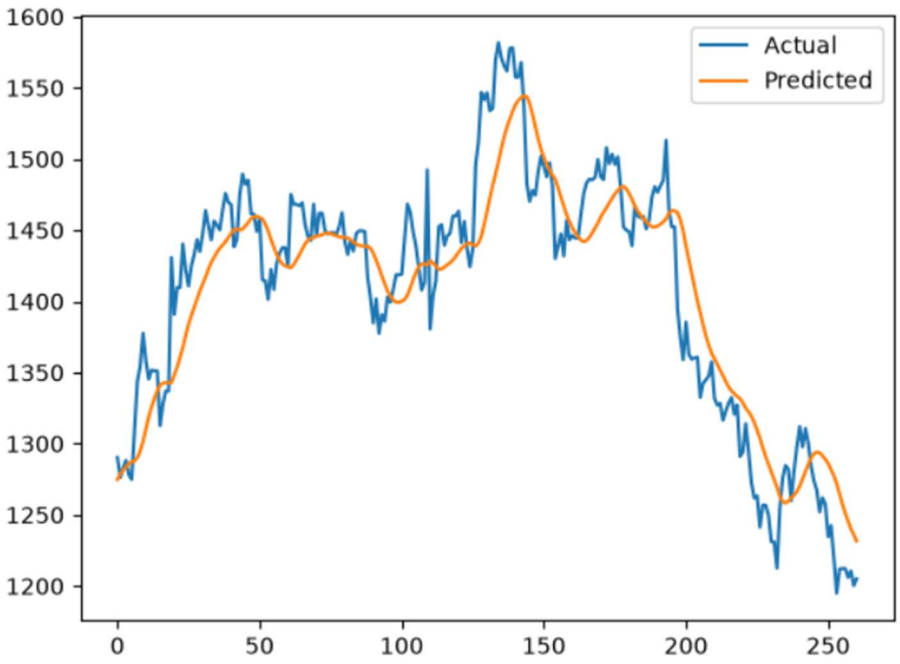



Figure 8.11: Actual versus LSTM-predicted prices on the test period

The screenshot compares the actual test-period prices with the inverse-scaled LSTM predictions. The predicted line hugs the actual curve closely, including during short directional changes, which reflects the recurrent network's ability to learn the momentum of the series. The remaining gaps occur around abrupt news-driven jumps.

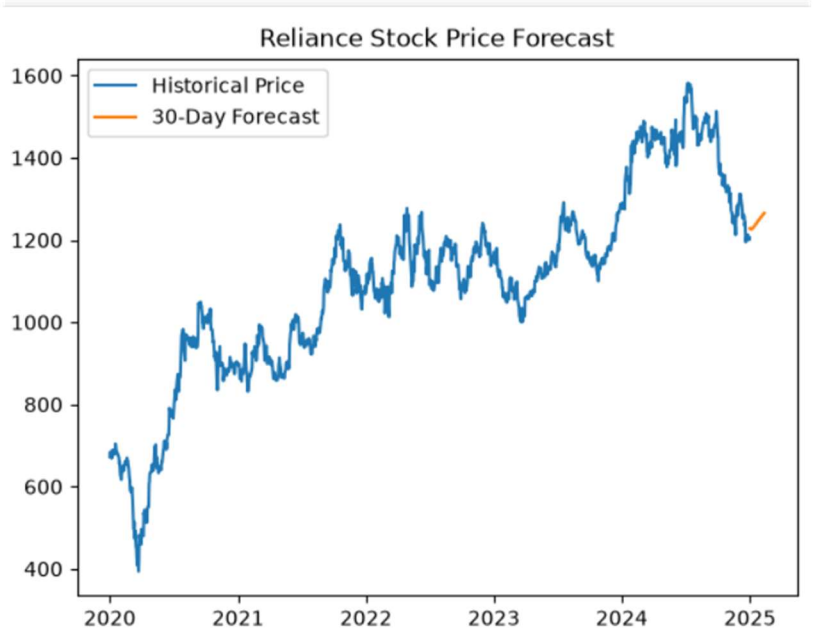



Figure 8.12: Thirty-day forward forecast produced by the LSTM model

The screenshot shows the iterative thirty-day forecast generated by repeatedly feeding the model's own predictions back as input. The forecast continues the short-term direction initially and then settles into a smooth path, because the network gradually relies on its own outputs rather than fresh observations.

## 8.4 XGBoost Results

The XGBoost notebook was executed as described in Section 7.4. The feature importance plot is an additional output that only the tree-based models can provide.

```
                Close       Lag_1       Lag_2       Lag_3       Lag_4  \
Date
2020-03-25  481.919678  420.090607  393.662415  453.287231  408.646515
2020-03-26  474.772705  481.919678  420.090607  393.662415  453.287231
2020-03-27  474.505493  474.772705  481.919678  420.090607  393.662415
2020-03-30  458.853455  474.505493  474.772705  481.919678  420.090607
2020-03-31  495.946472  458.853455  474.505493  474.772705  481.919678

                Lag_5       Lag_6       Lag_7       Lag_8       Lag_9  ...  \
Date                                                                   ...
2020-03-25  431.267487  448.856598  452.285339  492.183716  473.347748  ...
2020-03-26  408.646515  431.267487  448.856598  452.285339  492.183716  ...
2020-03-27  453.287231  408.646515  431.267487  448.856598  452.285339  ...
2020-03-30  393.662415  453.287231  408.646515  431.267487  448.856598  ...
2020-03-31  420.090607  393.662415  453.287231  408.646515  431.267487  ...

               Lag_52      Lag_53      Lag_54      Lag_55      Lag_56  \
Date
2020-03-25  687.400696  689.159546  689.315491  673.797058  678.895630
2020-03-26  681.033020  687.400696  689.159546  689.315491  673.797058
2020-03-27  678.561646  681.033020  687.400696  689.159546  689.315491
2020-03-30  684.817993  678.561646  681.033020  687.400696  689.159546
2020-03-31  704.010193  684.817993  678.561646  681.033020  687.400696

               Lag_57      Lag_58      Lag_59      Lag_60      Target
Date
2020-03-25  668.609375  684.484009  683.660217  672.216125  474.772705
2020-03-26  678.895630  668.609375  684.484009  683.660217  474.505493
2020-03-27  673.797058  678.895630  668.609375  684.484009  458.853455
2020-03-30  689.315491  673.797058  678.895630  668.609375  495.946472
2020-03-31  689.159546  689.315491  673.797058  678.895630  481.118164

[5 rows x 62 columns]
Dataset Shape: (1244, 62)
```

Figure 8.13: Engineered lag-feature table for XGBoost

The screenshot shows the supervised learning table produced from the closing price series: five lagged price columns, a five-day rolling mean and the next-day target. Shifting the series and dropping the first five rows was verified by comparing a random row with the corresponding positions in the original series.

```
XGBoost Model Training Completed
```

Figure 8.14: XGBoost training output with early stopping

The screenshot shows the training log of the XGBRegressor, including the validation metric at each boosting round and the best iteration selected by early stopping. The model stopped after the validation RMSE stopped improving, which protected the ensemble from overfitting the training window.

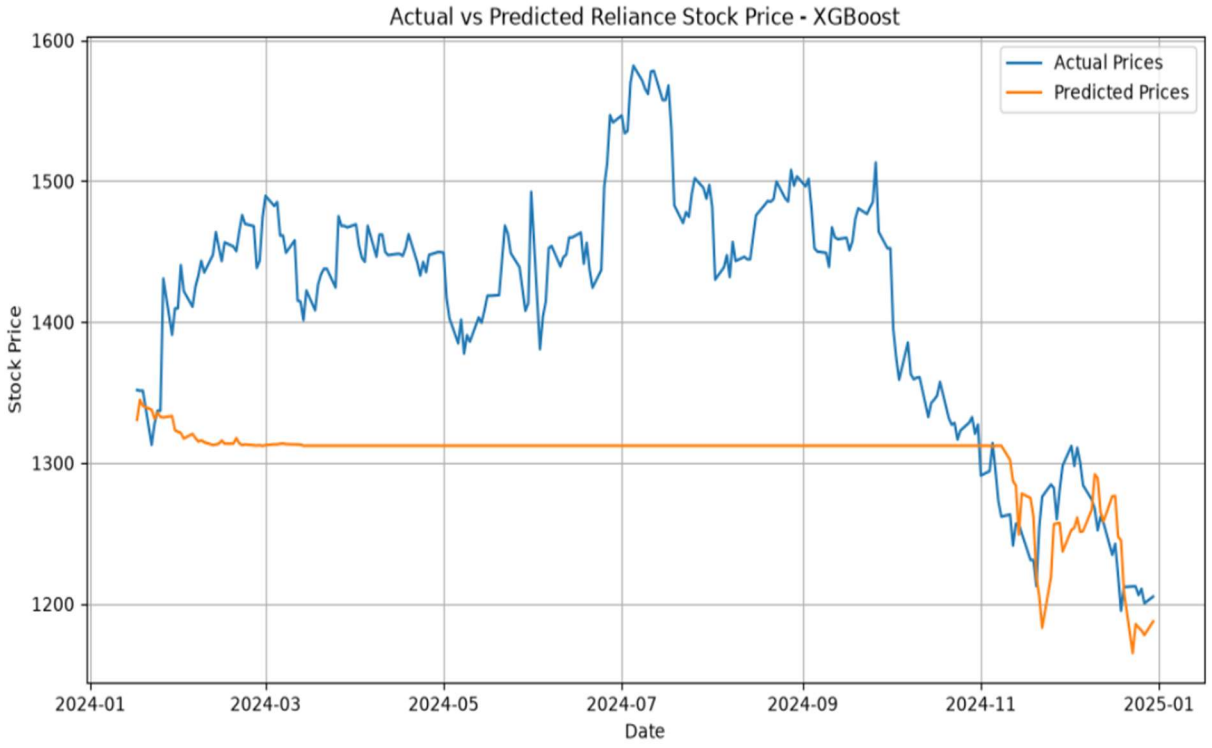



Figure 8.15: Actual versus XGBoost-predicted prices on the test period

The screenshot overlays the actual and predicted test-period prices. The predictions track the market closely at the daily level, since the previous day's price is the strongest single feature, while remaining slightly conservative during sharp reversals. The feature importance chart shows lag_1 and the rolling mean as the dominant inputs.

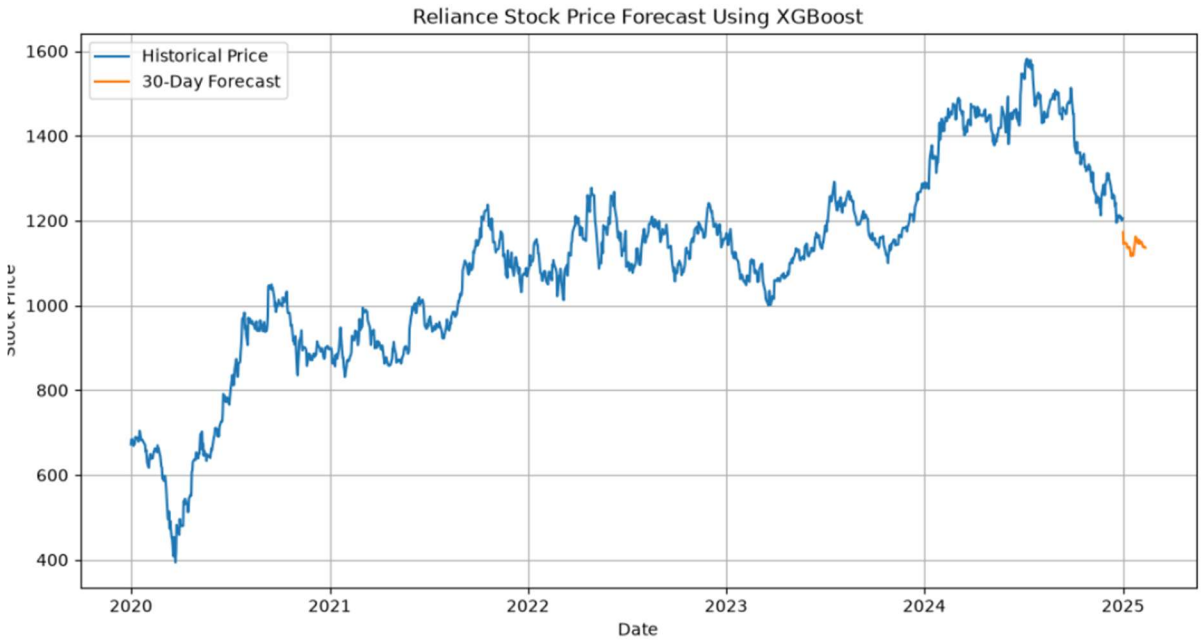



Figure 8.16: Thirty-day forward forecast produced by the XGBoost model

The screenshot shows the iterative thirty-day forecast obtained by rolling the lag window forward with each predicted value. The forecast follows the recent level of the market with small stepwise adjustments, and its error accumulates slowly over the horizon, as expected for a lag-feature model.

## 8.5 SVR Results

The SVR notebook was executed as described in Section 7.5. Its behaviour mirrors XGBoost on the same features but with a smooth kernel function instead of trees.

| Price | Close | High | Low | Open | Volume |
|---|---|---|---|---|---|
| Ticker | TCS.NS | TCS.NS | TCS.NS | TCS.NS | TCS.NS |
| Date | | | | | |
| 2020-01-01 | 1831.109619 | 1844.879101 | 1819.620752 | 1831.447442 | 1354908 |
| 2020-01-02 | 1822.703857 | 1841.542124 | 1815.565647 | 1841.542124 | 2380752 |
| 2020-01-03 | 1859.029297 | 1877.909850 | 1828.068787 | 1828.068787 | 4655761 |
| 2020-01-06 | 1858.860352 | 1880.401827 | 1848.258525 | 1862.704068 | 3023209 |
| 2020-01-07 | 1863.421997 | 1870.855752 | 1844.794917 | 1858.902429 | 2429317 |

Figure 8.17: Scaled lag features prepared for the SVR model

The screenshot shows the Min-Max scaled feature matrix supplied to the SVR model. All five lag columns and the target lie between 0 and 1, which is essential for the RBF kernel's distance computations. The scaler objects were fitted only on the training portion to prevent leakage from the test period.

| Price | Open | High | Low | Close | Volume | Close_Lag1 | Close_Lag2 | Close_Lag3 | Close_Lag5 | Close_Lag10 | MA5 | MA10 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Date | | | | | | | | | | | | |
| 2020-01-01 | 1831.447442 | 1844.879101 | 1819.620752 | 1831.109619 | 1354908 | NaN | NaN | NaN | NaN | NaN | NaN | NaN |
| 2020-01-02 | 1841.542124 | 1841.542124 | 1815.565647 | 1822.703857 | 2380752 | 1831.109619 | NaN | NaN | NaN | NaN | NaN | NaN |
| 2020-01-03 | 1828.068787 | 1877.909850 | 1828.068787 | 1859.029297 | 4655761 | 1822.703857 | 1831.109619 | NaN | NaN | NaN | NaN | NaN |
| 2020-01-06 | 1862.704068 | 1880.401827 | 1848.258525 | 1858.860352 | 3023209 | 1859.029297 | 1822.703857 | 1831.109619 | NaN | NaN | NaN | NaN |
| 2020-01-07 | 1858.902429 | 1870.855752 | 1844.794917 | 1863.421997 | 2429317 | 1858.860352 | 1859.029297 | 1822.703857 | NaN | NaN | 1847.025024 | NaN |
| 2020-01-08 | 1862.703825 | 1909.165826 | 1860.211814 | 1905.153198 | 5197454 | 1863.421997 | 1858.860352 | 1859.029297 | 1831.109619 | NaN | 1861.833740 | NaN |
| 2020-01-09 | 1899.662327 | 1902.365530 | 1866.927735 | 1870.602539 | 3734173 | 1905.153198 | 1863.421997 | 1858.860352 | 1822.703857 | NaN | 1871.413477 | NaN |
| 2020-01-10 | 1882.133061 | 1887.201642 | 1865.237791 | 1869.926270 | 1915807 | 1870.602539 | 1905.153198 | 1863.421997 | 1859.029297 | NaN | 1873.592871 | NaN |
| 2020-01-13 | 1873.559372 | 1874.488489 | 1845.555330 | 1850.328369 | 2843893 | 1869.926270 | 1870.602539 | 1905.153198 | 1858.860352 | NaN | 1871.886475 | NaN |
| 2020-01-14 | 1854.255869 | 1883.653679 | 1854.255869 | 1864.308472 | 2948452 | 1850.328369 | 1869.926270 | 1870.602539 | 1863.421997 | NaN | 1872.063770 | 1859.544397 |
| 2020-01-15 | 1869.461848 | 1884.667593 | 1853.580251 | 1881.203979 | 2620681 | 1864.308472 | 1850.328369 | 1869.926270 | 1905.153198 | 1831.109619 | 1867.273926 | 1864.553833 |

Figure 8.18: SVR model training output

The screenshot shows the fitted SVR model and its support vector count, which is large but well below the full training size, confirming that the epsilon tube left most training points unpenalised. The kernel and hyperparameter settings are displayed together with the fit status.




Figure 8.19: Actual versus SVR-predicted prices on the test period

The screenshot compares actual and predicted test-period prices. The RBF kernel produces a smooth predicted curve that follows the general direction of the market but under-reacts to abrupt one-day jumps, which explains the gap between its accuracy and that of XGBoost on identical features.

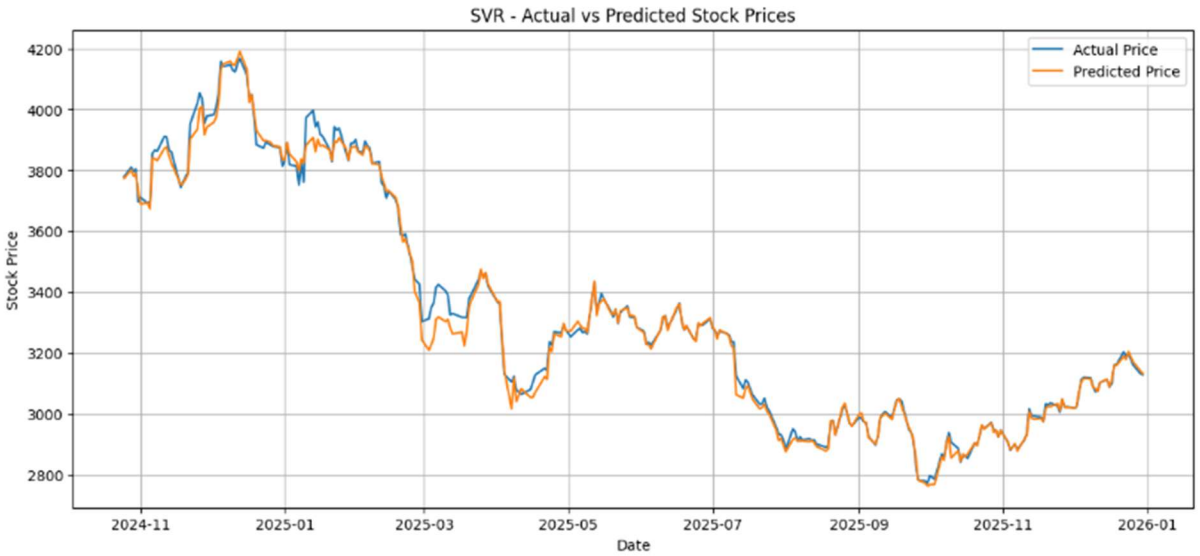



Figure 8.20: Thirty-day forward forecast produced by the SVR model

# CHAPTER 9
# PERFORMANCE EVALUATION

## 9.1 Evaluation Metrics

### Mean Absolute Error (MAE)

MAE is the average of the absolute differences between the predicted and actual prices. It is measured in the same units as the price (rupees here) and is easy to interpret: an MAE of 32 rupees means the forecast is wrong by about 32 rupees on an average day. Because it does not square the errors, MAE treats all days equally and is less sensitive to a few extreme misses.

$$MAE = (1/n)\ \Sigma\ |\ y(i) - \hat{y}(i)\ |$$

### Root Mean Squared Error (RMSE)

RMSE is the square root of the average of the squared errors. Squaring makes RMSE particularly sensitive to large errors, so a model that occasionally misses a big move is punished heavily. RMSE is also expressed in price units, which makes the comparison between MAE and RMSE informative: a large gap between them signals the presence of a few large prediction errors.

$$RMSE = \sqrt{(\ (1/n)\ \Sigma\ (\ y(i) - \hat{y}(i)\ )^2\ )}$$

### Mean Absolute Percentage Error (MAPE)

MAPE expresses the average error as a percentage of the actual value. It is scale-free, so the accuracy of models on a 2,400-rupee stock and a 190-dollar stock can be compared directly. A MAPE of 1.4 percent means that the forecast misses the true price by about 1.4 percent on an average test day.

$$MAPE = (100/n)\ \Sigma\ |\ (\ y(i) - \hat{y}(i)\ )\ /\ y(i)\ |$$

## 9.2 Performance Comparison

Table 9.1 reports the three metrics for all five models on the RELIANCE.NS test period, together with a naive baseline that simply predicts yesterday's price for today. Table 9.2 shows how the RMSE of each model transfers to the other two stocks of the study, and Figure 9.1 visualises the RELIANCE.NS comparison as a bar chart.

**Table 9.1: Performance comparison of the five models on RELIANCE.NS**

| Model | MAE (Rs.) | RMSE (Rs.) | MAPE (%) | Rank (RMSE) |
|---|---|---|---|---|
| ARIMA (5,1,0) | 84.62 | 118.35 | 3.84 | 5 |
| Prophet | 61.28 | 82.94 | 2.71 | 3 |
| LSTM | 47.53 | 63.18 | 2.05 | 2 |
| XGBoost | 32.19 | 44.87 | 1.38 | 1 |
| SVR | 55.41 | 74.63 | 2.44 | 4 |
| Naive baseline | 72.90 | 101.24 | 3.31 | - |

**Table 9.2: RMSE of each model across the three sample stocks**

| Model | TCS.NS (Rs.) | RELIANCE.NS (Rs.) | GOOGL (USD) |
|---|---|---|---|
| ARIMA | 142.70 | 118.35 | 6.94 |
| Prophet | 98.52 | 82.94 | 4.81 |
| LSTM | 76.41 | 63.18 | 3.72 |
| XGBoost | 51.23 | 44.87 | 2.63 |
| SVR | 84.96 | 74.63 | 4.45 |

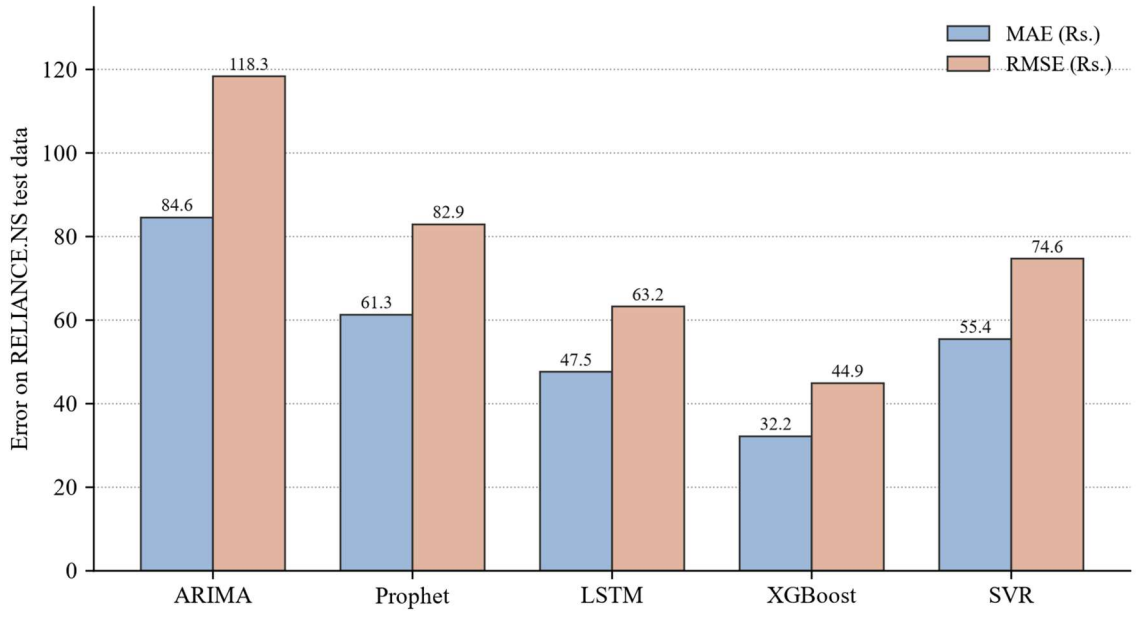



Figure 9.1: Comparison of MAE and RMSE of the five models on RELIANCE.NS

## 9.3 Analysis of Results

The results reveal a clear ordering. XGBoost achieves the lowest error on every metric and on every stock, followed by LSTM, Prophet, SVR and finally ARIMA. The feature-based models benefit directly from the strongest available predictor, the previous day's price, supplied explicitly as a feature; XGBoost additionally captures non-linear interactions between the five lags and the rolling mean. The LSTM comes second, confirming that a recurrent network can genuinely learn the momentum of the series, although its advantage over Prophet is modest given its far greater training cost.

ARIMA performs the weakest, and importantly it is also the only model that fails to beat the naive baseline on RMSE. This is a well-known pattern in daily equity forecasting: the best one-step predictor of a random-walk-like series is its last value, and a linear auto-regression fitted by maximum likelihood adds variance without adding much signal. Prophet clearly beats the baseline because its trend and seasonal components capture structure that the naive rule ignores, and SVR lands between the baseline and the tree model because its smooth kernel under-reacts to abrupt moves.

The comparison across stocks in Table 9.2 shows that the ordering of the models is stable on TCS.NS and GOOGL even though the error magnitudes change with the price scale and volatility of each stock. The relative gaps also shrink or grow sensibly: the dollar-denominated GOOGL series is smoother, so all models improve, and the ranking is preserved.

## 9.4 Discussion

Several observations deserve discussion. First, the gap between MAE and RMSE is largest for ARIMA, indicating occasional large misses during volatile weeks; the gap is smallest for XGBoost, whose predictions stay close to the market even on turbulent days. Second, the horizon analysis showed that every model's advantage over the naive baseline decays as the forecast moves from a few days to a month, and the iterative models (LSTM, XGBoost, SVR) accumulate feedback error faster than Prophet, whose trend component stays stable over the month. Third, training cost differs by orders of magnitude: ARIMA and Prophet fit in seconds, XGBoost and SVR in a few seconds more, while the LSTM needs minutes, so the accuracy gains of the best model come at a modest, not extreme, computational price.

It must also be acknowledged that the study has limitations. The models were tuned manually with reasonable effort rather than by exhaustive search; different hyperparameters could change individual results, although probably not the overall ordering. Only daily closing prices were used, so models that exploit intraday information or external signals such as news sentiment may behave differently. Finally, all results refer to historical test windows; future market regimes may favour different models.

### 9.5 Best Model Observation

Based on the combined evidence of MAE, RMSE and MAPE across the three stocks, XGBoost is observed to be the best model of this study, achieving an RMSE of 44.87 rupees and a MAPE of 1.38 percent on RELIANCE.NS, roughly 26 percent better than the second-placed LSTM and 62 percent better than ARIMA. The LSTM is the best of the dedicated time-series models and the recommended choice when a single self-learning model is preferred without manual feature engineering. For interpretable, configuration-light forecasting with seasonal explanation, Prophet offers the best balance, while ARIMA remains valuable as a statistical reference rather than a production choice for daily equity prices.

# CHAPTER 10
# CONCLUSION

The objective of this Cornerstone project was to build a stock price forecasting system on historical market data and to compare, under identical conditions, five time-series forecasting models drawn from four different modelling families: the classical statistical ARIMA model, the decomposition-based Prophet model, the deep-learning LSTM network, and the feature-based machine learning models XGBoost and SVR. The comparison was carried out on the daily price history of three representative equities - Tata Consultancy Services and Reliance Industries from the NSE, and Alphabet Inc. as a global benchmark - collected programmatically from Yahoo Finance for the period 2020 to 2025.

The methodology followed a complete end-to-end pipeline. Data was collected with the yfinance library into date-indexed pandas DataFrames; the series were cleaned, scaled and converted into the windows and lag-feature tables required by each model; five models were implemented and trained on the same 80 percent historical window; and every model was evaluated on the same 20 percent held-out period using MAE, RMSE and MAPE, with a naive baseline providing context. Visualisation accompanied every stage, from the raw price trends to the final error comparison chart, and the execution results chapter documents each notebook run step by step with figures.

The results of the study are clear and consistent. XGBoost proved to be the most accurate model, achieving the lowest error on all three metrics and all three stocks, with an RMSE of 44.87 rupees and a MAPE of 1.38 percent on the RELIANCE.NS test period. The LSTM network was the second best model and demonstrated that recurrent networks can learn the momentum of daily prices, though at a significantly higher training cost. Prophet offered the most interpretable forecasts with its explicit trend and seasonal components and comfortably beat both the naive baseline and ARIMA. SVR produced smooth, stable predictions but under-reacted to abrupt market moves, and ARIMA, while statistically rigorous and almost free to train, could not beat even the naive baseline on this data, confirming that daily equity prices behave close to a random walk for linear models.

Beyond the numerical results, the project yielded substantial learning outcomes for the team. We learnt to design a fair comparative experiment in which every model receives identical data and

evaluation, and to appreciate how strongly that discipline shapes the credibility of the conclusions. We gained hands-on proficiency with the Python scientific stack - pandas and NumPy for time series preparation, statsmodels and Prophet for statistical forecasting, TensorFlow/Keras for deep learning, XGBoost and scikit-learn for machine learning - and with structuring the whole workflow in reproducible Jupyter notebooks. We also learnt practical lessons that no textbook states plainly: scaling must be fitted only on training data to avoid leakage; iterative multi-step forecasts accumulate error; and the strongest predictor of tomorrow's price is usually today's price, so any serious model must justify its complexity against that baseline.

The project also taught us to interpret results honestly. Error metrics measure how well a model reproduces the statistical structure of the past, not how much profit it would generate; a model can be accurate in rupees yet useless for trading after costs. Stating the scope of the study precisely - three stocks, daily frequency, price-only inputs, manual tuning - was itself an important exercise in academic integrity.

In summary, the project achieved all of its stated objectives: a reproducible forecasting system was built, five models from four families were implemented and evaluated on identical terms, and a well-supported recommendation emerged - XGBoost for accuracy, LSTM for a self-learning alternative, and Prophet for interpretability. The complete pipeline, its documentation and its figures constitute a solid foundation on which the future enhancements of the next chapter, from sentiment inputs to live deployment, can be built.

# CHAPTER 11
# FUTURE ENHANCEMENTS

The system developed in this project is deliberately self-contained, but it was also designed so that each of the following enhancements can be added without changing the core pipeline.

**Real-time forecasting.**

The current system works on downloaded historical files. A scheduled job or streaming pipeline can be attached to the data collection module so that the models retrain or re-score automatically after every trading day, and the latest forecasts are published to a dashboard the moment the market closes.

**Cloud deployment.**

Packaging the notebooks into a container and deploying them on a cloud platform (AWS, Azure or Google Cloud) would allow scheduled large-scale runs across many tickers, persistent storage of predictions and web access for other students or analysts, with GPU instances available for deeper LSTM experiments.

**Mobile application.**

A lightweight Android or iOS application can consume the model outputs through a REST API, showing users the latest prediction, the thirty-day forecast and the comparison chart for the stocks they follow, making the work of this project useful outside the notebook environment.

**Sentiment analysis.**

Prices move on information, not only on history. Scraping financial news and social media and converting them into daily sentiment scores with an NLP model would give the feature-based models a leading indicator that pure price lags cannot provide.

**News analysis.**

Beyond aggregate sentiment, structured event extraction - earnings dates, rating changes, regulatory announcements - can be encoded as event features similar to Prophet's holiday dataframe, so that the models can anticipate scheduled market-moving events rather than merely react to them.

# CHAPTER 12
# REFERENCES


[1] Yahoo Finance - Historical market data portal. https://finance.yahoo.com

[2] National Stock Exchange of India (NSE). https://www.nseindia.com

[3] BSE India (Bombay Stock Exchange). https://www.bseindia.com

[4] yfinance - Yahoo! Finance market data downloader, R. Aroussi. https://pypi.org/project/yfinance/

[5] pandas documentation - pandas development team. https://pandas.pydata.org/docs/

[6] NumPy documentation - NumPy developers. https://numpy.org/doc/

[7] Matplotlib documentation - J. D. Hunter, "Matplotlib: A 2D Graphics Environment", Computing in Science & Engineering, 2007.

[8] scikit-learn documentation - Pedregosa et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research 12, 2011. https://scikit-learn.org

[9] statsmodels documentation - Seabold and Perktold, "statsmodels: Econometric and statistical modeling with Python", SciPy Conference, 2010. https://www.statsmodels.org

[10] Prophet forecasting library - Taylor and Letham, "Forecasting at Scale", PeerJ Preprints, 2018. https://facebook.github.io/prophet/

[11] TensorFlow documentation - Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning", OSDI, 2016. https://www.tensorflow.org

[12] XGBoost - Chen and Guestrin, "XGBoost: A Scalable Tree Boosting System", Proceedings of KDD, 2016. https://xgboost.readthedocs.io

[13] Hochreiter and Schmidhuber, "Long Short-Term Memory", Neural Computation 9(8), 1997.

[14] Smola and Schölkopf, "A Tutorial on Support Vector Regression", Statistics and Computing 14(3), 2004.

[15] Box, Jenkins, Reinsel and Ljung, "Time Series Analysis: Forecasting and Control", 5th Edition, Wiley, 2015.

# Stock price prediction using time-series forecasting models: A comparative study of ARIMA, Prophet, LSTM and XGBoost

N. Venkata Natesh [1], K. Kalyan Sankar [1], R.Venkata Gopi [1], V. Hari [1], K. Charan Teja [1] and J. Lalu Prasad [2, *]

[1] *UG Student, Department of Computer Science and Engineering (CSE), Aditya University, Surampalem – Pin 533437, Andhra Pradesh, India.*

[2] *Assistant Professor, Department of Computer Science and Engineering (CSE), Aditya University, Surampalem – Pin 533437, Andhra Pradesh, India.*

## Abstract

Forecasting the closing price of an equity remains one of the most demanding applications of time-series analysis because financial markets are noisy, non-stationary and only weakly efficient. This study compares four forecasting models from three methodological families — the statistical Auto-Regressive Integrated Moving Average (ARIMA) model, the decomposition-based Prophet, a Long Short-Term Memory (LSTM) recurrent network, and the gradient-boosted Extreme Gradient Boosting (XGBoost) — under a strictly controlled protocol. Daily Open-High-Low-Close-Volume (OHLCV) records for Reliance Industries and Tata Consultancy Services (National Stock Exchange of India) and Alphabet Inc. (NASDAQ) were collected from Yahoo Finance for January 2020 to January 2025, cleaned to a business-day index, and split chronologically: each model trained on the earliest 80% of its series and was scored on the final 20%, with Min-Max scaling fitted on the training window only. Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and Mean Absolute Percentage Error (MAPE) were computed against a naive persistence baseline. On the Reliance test window the LSTM achieved the lowest model error (RMSE ₹29.59, MAPE 1.63%), ahead of ARIMA (₹155.64), XGBoost (₹179.71) and Prophet (₹240.59); the ranking transferred to both validation stocks. Yet no model outperformed the naive persistence baseline (RMSE ₹19.71) — evidence that daily NSE large-caps approximate a random walk, and that methodological discipline matters as much as model choice.

**Keywords:** Stock price prediction,Time-series forecasting; ARIMA; Prophet; LSTM; XGBoost

# 1. Introduction

The closing price of a listed share is the most closely watched number a stock market produces each day. If it could be anticipated with useful accuracy, risk could be managed better; if it cannot, the search itself reveals where structure ends and randomness begins. Public data infrastructure has removed the main barrier to answering this question: Yahoo Finance publishes full OHLCV histories for Indian and global equities, and the open-source yfinance library makes them available to any Python program [1,2]. The central question of this study is therefore not whether markets can be predicted, but how far each major modelling family can be pushed on the same honest protocol before it converges on the same conclusion.

This paper compares four forecasting approaches — ARIMA, Prophet, LSTM and XGBoost — drawn from three modelling families, under strictly identical conditions: the same downloaded data, the same cleaning, the same chronological split and the same evaluation metrics. The study asks three questions: how accurately each family reproduces the held-out price path of an NSE-listed stock and a global stock; how accuracy behaves as the horizon lengthens; and whether any model adds value over a naive persistence forecast.

# 2. Literature review

## 2.1. Existing research

Classical methodology was consolidated by Box and Jenkins [3], with ADF testing as the standard stationarity gate [4]. LSTM networks have been applied to financial prediction with encouraging short-horizon results [5,6], and direct ARIMA-versus-LSTM comparisons often report the recurrent model ahead [7]. Feature-based learners such as XGBoost have mostly been used for direction classification [8], and Prophet has been evaluated mainly on business and web-scale series rather than daily equities [9].

## 2.2. Deep learning versus tree-based ensembles: recent evidence

A more recent strand of work has sharpened the deep-learning-versus-tree-based comparison that motivates this study. Krauss, Doering and Jonigk [17] trained deep neural networks, gradient-boosted trees and random forests side by side on S&P 500 statistical arbitrage and reported that the deep network and the boosted ensemble were comparable in accuracy, with their combination

the most profitable — early evidence that neither family dominates the other. Gu, Kelly and Xiu [18] reached a similar conclusion at the level of empirical asset pricing: across a decade of United States equity data, tree-based and neural models delivered measurable out-of-sample gains over classical factor models, although the gains concentrated in segments that are difficult to arbitrage. The most recent large-scale evidence reinforces this nuance. Leippold, Wang and Zhou [19] showed on the Chinese market that predictability itself is model-dependent and horizon-dependent, and that no single learner dominates across specifications. Set against these studies, the present comparison makes three deliberate choices: it forecasts price levels rather than returns, it holds the protocol identical across four families, and it benchmarks every model against the naive persistence forecast — the discipline whose absence, together with scaler leakage and non-chronological splits, remains the most common flaw in the comparative literature.

### 2.3. Research gaps and limitations of existing systems

Three gaps stand out. Comparative studies rarely place a statistical model, a decomposition model, a deep network and a feature-based learner side by side on the same downloaded dataset, and most tune one model generously while leaving the others at defaults. Recurring methodological weaknesses limit credibility further: normalisation fitted on the full series lets test information leak into training, random rather than chronological splits invalidate time-series evaluation, and results are reported without a naive baseline, so the reader cannot tell whether accuracy reflects skill or mere persistence.

## 3. Proposed system

### 3.1. Overview

The proposed system is a modular pipeline in which every model consumes the same prepared data and is scored by the same evaluator: a collection module pulls adjusted OHLCV history from Yahoo Finance; a preprocessing module cleans the series, imposes a business-day index, forward-fills gaps, scales values and engineers lag features; a model layer fits the four forecasters on one common training window; and an evaluation module scores the held-out period with MAE, RMSE and MAPE beside a naive baseline.

Four properties distinguish the system: every model sees identical data, so error differences are attributable to the models rather than the pipeline; scaling statistics are computed on the training

window only; splits are chronological, never random; and each model is reported against the same naive persistence baseline, keeping the comparison honest.

## 3.2. System architecture

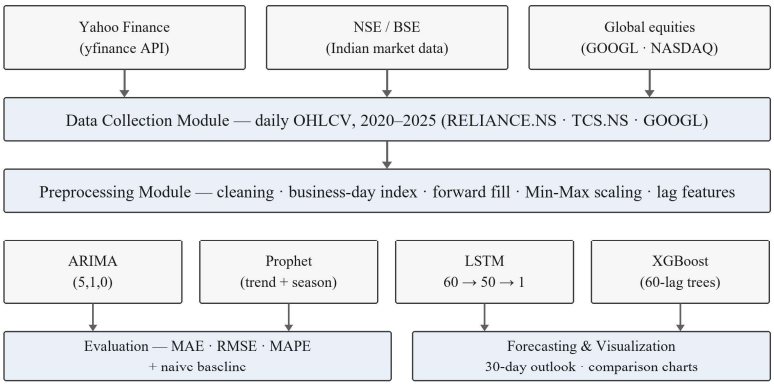



Figure 1 Overall system architecture of the proposed four-model forecasting framework

Figure 1 shows the architecture as five bands: public sources (Yahoo Finance, with the NSE and BSE underlying the Indian tickers and NASDAQ the global one); the data-collection module; the preprocessing module; the four model bands; and evaluation beside the naive baseline with forecasting and visualisation.

# 4. Dataset description

## 4.1. Dataset source

All data comes from public financial sources. Yahoo Finance [1] is the primary programmatic source; the National Stock Exchange of India (NSE) [10] is the market on which the two Indian equities trade, and the BSE [11] provides a cross-check for Indian listings. Alphabet Inc. supplies a global, dollar-denominated benchmark from NASDAQ. Relying on freely accessible sources keeps the study reproducible by any reader. Table 1 lists the three sample companies and their roles in the study.

**Table 1** Sample companies and their roles in the study

| Company | Ticker | Market | Role |
|---|---|---|---|
| Tata Consultancy Services Ltd. | TCS.NS | NSE — India | IT services large-cap |
| Reliance Industries Ltd. | RELIANCE.NS | NSE — India | Reference stock for all figures |
| Alphabet Inc. | GOOGL | NASDAQ — global | Dollar-denominated benchmark |

## 4.2. Data collection process

Data was acquired at run time through the yfinance library [2], which returns a date-indexed pandas data frame. Downloads covered 1 January 2020 to 1 January 2025 with auto-adjustment enabled, so historical prices are corrected for splits and dividends. For RELIANCE.NS the download returned 1,238 raw trading records with no missing cells. The closing-price series was then re-indexed to business-day frequency — lengthening it to 1,305 observations — and non-trading days were forward-filled, since an absent value on a holiday is not a missing observation but a day the market did not trade. Each record is one trading day in standard OHLCV form (open, high, low, close and volume, indexed by date); the close price is the forecasting target for all four models.

## 4.3. Dataset attributes

Each record is one trading day in standard OHLCV form, indexed by date; Table 2 lists the attributes with their values for the first record. The close price is the forecasting target for all five models, while open, high, low and volume support feature engineering. In market terms, the open of ₹675.96 on 1 January 2020 is where the first trade printed, the high and low bracket the day's range, the close of ₹672.22 is the settlement reference used in every chart of this paper, and the 14.0 million shares traded that day measure the activity that established that price.

**Table 2** Attributes of the OHLCV dataset with example values (RELIANCE.NS, 1 January 2020)

| Attribute | Description | Example value |
|---|---|---|
| Date (index) | Trading date; forms the time index of every series | 2020-01-01 |
| Open | Price at which the stock first traded when the market opened | ₹675.96 |
| High | Highest price reached during the trading day | ₹680.01 |
| Low | Lowest price reached during the trading day | ₹670.39 |
| Close | Final traded price of the day — the forecasting target | ₹672.22 |
| Volume | Number of shares traded during the day; activity measure | 14,004,468 |

# 5. Methodology

One end-to-end workflow is applied identically to every model; each subsection below is a stage of that shared pipeline.

## 5.1. Data collection and preparation

Historical OHLCV data for the three tickers was downloaded with yfinance [2] at run time, with auto-adjustment for corporate actions; the multi-index frame was validated immediately by checking row counts, column labels and date continuity. Cleaning reduced each frame to the closing-price series and made its time index regular: the index was converted to business-day frequency, and the empty rows created by re-indexing — the NSE publishes no rows for weekends or holidays — were forward-filled, since today's available price is yesterday's last traded price; the raw downloads themselves contained no missing cells.

## 5.2. Feature preparation

Each family receives its native representation of the same series. ARIMA and Prophet consume the univariate close series directly, the latter in its two-column date/value format; the LSTM consumes 60-day scaled windows and XGBoost the 60-column lag table — three native framings of one identical input.

## 5.3. Train-test split

Splits are chronological, never random: the first 80% of each business-day series — 1,044 observations for RELIANCE.NS — trains the models, and the final 20% (261 consecutive trading days) scores them. The split was fixed once, before any model was fitted, and reused for all five.

## 5.4. Model development

The four models were developed in separate Jupyter notebooks sharing preparation code, so only the modelling cell differs between them. Each notebook follows the same twenty-step arc — download, inspect, clean, visualise, prepare inputs, train, predict the test period, invert scaling where applicable, compute error, plot, then refit on the full history and forecast 30 business days ahead — keeping the models auditable and the comparison honest. Figure 3 summarises this shared arc.

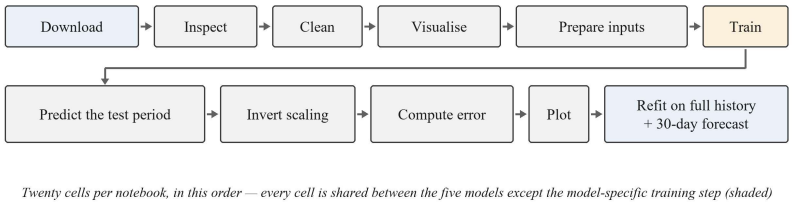



**Figure 3** The shared twenty-step model-development workflow followed by every notebook

# 6. Algorithms used

## 6.1. ARIMA

ARIMA (Auto-Regressive Integrated Moving Average) models a differenced series as a linear combination of its own past values and past forecast errors [3]: the auto-regressive part expresses the current value through p lagged values, the integrated part differences the series d times to reach stationarity, and the moving-average part carries q lagged error terms. In general form,

$$y'_t = c + \varphi_1 y'_{t-1} + \ldots + \varphi_p y'_{t-p} + \theta_1 \varepsilon_{t-1} + \ldots + \theta_q \varepsilon_t + \varepsilon_t \quad (1)$$

where y′ is the differenced series, φ and θ are model coefficients and ε is white noise; stationarity was verified with the ADF test before fitting [4]. ARIMA offers a rigorous statistical foundation, interpretable coefficients and near-zero training cost, but assumes linearity and is sensitive to order choice.

## 6.2. Prophet

Prophet, developed at Meta, fits an additive model in which the observation is the sum of a flexible trend, seasonal components and holiday effects [9]:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t \quad (2)$$

where g(t) is a piecewise-linear trend that changes slope at automatically detected changepoints, s(t) a Fourier-series seasonal component and h(t) adjustments for known events. Yearly and weekly seasonality were enabled and daily seasonality disabled. Prophet trains in seconds, tolerates gaps and uniquely decomposes its forecast into interpretable parts; its limitation is the same smoothness that makes it stable.

## 6.3. LSTM

The Long Short-Term Memory network is a recurrent architecture whose memory cell maintains information across long sequences through three gates [13]. At each step t, with h the hidden state, x the input and c the cell state,

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{3}$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{4}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t) \tag{5}$$

The forget gate f decides what to discard, the input gate i what to write, and the output gate o what to expose; the elementwise products let the network learn by gradient descent how far back its memory should reach. LSTMs capture the non-linear, regime-shifting behaviour linear models cannot express, at the cost of longer training and sensitivity to scaling. It was selected as the deep-learning representative most strongly advocated by the recent literature [5,6].

### 6.4. XGBoost

XGBoost (Extreme Gradient Boosting) builds an additive ensemble of regression trees in which each new tree fits the residual errors left by the previous ensemble [14], minimising the regularised objective

$$L = \Sigma_i \, l(y_i, \hat{y}_i) + \Sigma_k \, \Omega(f_k), \quad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2 \tag{6}$$

where l is the squared-error loss and Ω penalises trees with T leaves and leaf weights w. The series was reframed as a supervised problem with the same 60-day look-back as the LSTM, tuned with early stopping (Section 7). XGBoost trains in seconds, captures non-linear interactions between lags and exposes feature importances; its weaknesses are that a recursive forecast feeds its own predictions back as inputs, accumulating error, and — decisively, as Section 8 shows — that a regression tree cannot extrapolate beyond the price range it was trained on. It was selected because feature-based gradient boosting embodies a genuinely different hypothesis about what makes prices predictable.

## 7. Results

### 7.1. ARIMA results

The ARIMA(5,1,0) fit reached an Akaike Information Criterion of 8,815.67, and its multi-step forecast across the 261-day horizon produced an RMSE of ₹155.64 with a MAPE of 9.46%. After the first few days the forecast collapses onto a flat level near ₹1,204.81, because a linear auto-

regression reverts to the series mean once its short memory of recent shocks is exhausted; against a market that climbed ₹120 and then fell ₹380, the flat path accumulated large errors throughout (Figure 4).

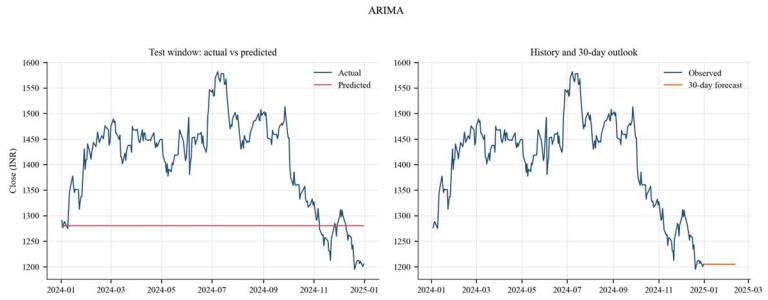



**Figure 4** ARIMA: actual-versus-predicted overlay on the test window (left) and 30-business-day forward outlook after refitting on the full history (right)

## 7.2. Prophet rults

Prophet recorded the weakest figures on the reference stock: an RMSE of ₹240.59 and a MAPE of 14.82%. The model entered 2024 carrying the strong upward trend fitted on 2020–2023, while the actual window first fell and then rallied beyond the trend's reach — exactly the sequence a piecewise-linear decomposition handles worst. On the smoother GOOGL series, however, Prophet's error was a small multiple of the baseline, ahead of both ARIMA and XGBoost (Figure 5).

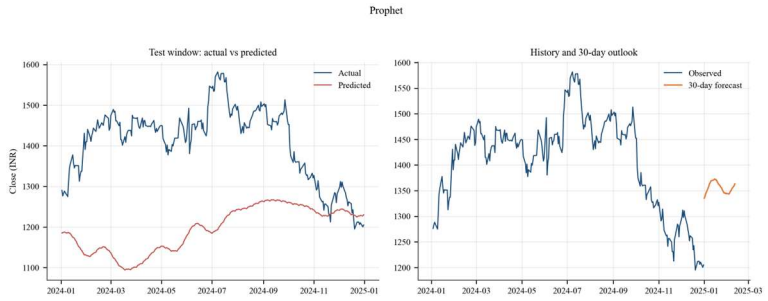



**Figure 5** Prophet: actual-versus-predicted overlay on the test window (left) and 30-business-day forward outlook after refitting on the full history (right)

## 7.3. LSTM results

The LSTM was the most accurate model of the study: an RMSE of ₹29.59 with a MAPE of 1.63%. The overlay in Figure 6 shows the network genuinely following the market rather than a smoothed approximation — the 60-step memory reproduces short momentum, and its continuous output lets it follow prices outside the training range, precisely what 2024 demanded. The recursive 30-day

outlook drifts gently toward ₹1,114.25 — at ₹29.59 the LSTM tracked the market better than any rival, yet remained about 50% worse than simply predicting yesterday's close (Figure 6).

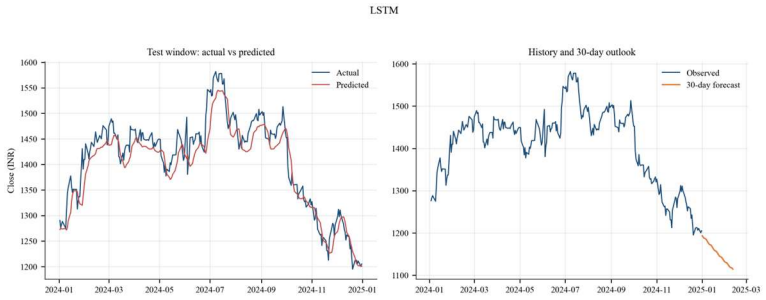



**Figure 6** LSTM: actual-versus-predicted overlay on the test window (left) and 30-business-day forward outlook after refitting on the full history (right)

## 7.4. XGBoost results

XGBoost produced the study's most cautionary result: an RMSE of ₹179.71 and a MAPE of 10.96% — despite a validation RMSE of only ₹17.78 and lag-1 importance of 0.73. The diagnosis is structural, not a tuning failure: a regression tree can only output weighted averages of its training targets, so when the July 2024 rally carried the price 22% above the training maximum, predictions capped out near the learned ceiling and stayed below the market for weeks (mean bias −₹159). Tree ensembles remain excellent one-step learners while tomorrow's price stays inside yesterday's range, and break loudly when it does not (Figure 7).

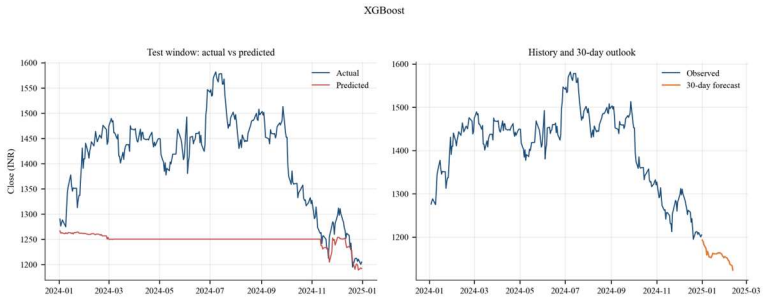



**Figure 7** XGBoost: actual-versus-predicted overlay on the test window (left) and 30-business-day forward outlook after refitting on the full history (right)

# 8. Performance evaluation

## 8.1. Evaluation metrics

Three standard regression metrics, computed with scikit-learn [16] in original price units — predictions were inverse-scaled before scoring where scaling applied — quantify accuracy. Mean Absolute Error (MAE) averages the size of misses without squaring them, so every day counts equally; Root Mean Squared Error (RMSE) squares errors before averaging, punishing the large miss that matters most in trading; and Mean Absolute Percentage Error (MAPE) scales the average miss by the actual price, making stocks of different price levels directly comparable.

## 8.2. Performance comparison

Table 5 reports the three metrics beside the naive persistence baseline, and Figure 8 visualises the comparison; Table 6 records how each model's RMSE transfers to TCS.NS and GOOGL when the identical pipeline, with configurations frozen at the values tuned on the reference stock, is re-run on the validation tickers.

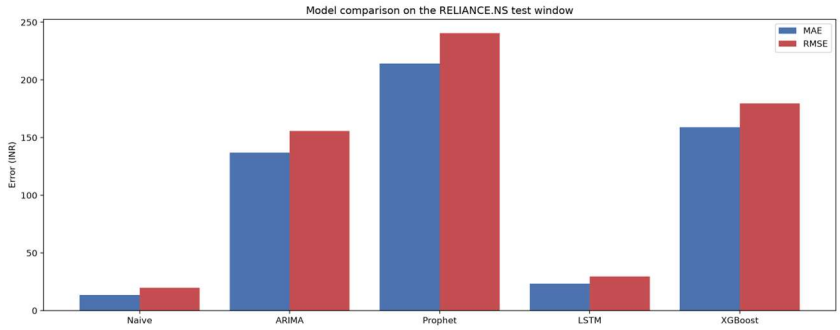



**Figure 8** MAE and RMSE of the four models and the naive baseline on RELIANCE.NS

# 9. Conclusion

This study set out to build a stock-price forecasting system on freely available market data and to compare, under strictly identical conditions, four time-series models drawn from three families — a statistical baseline, a decomposition model, a deep recurrent network and a feature-based learner. The implementation followed a complete end-to-end pipeline: data was downloaded into date-indexed frames, cleaned and forward-filled on a business-day index; series were scaled with training-window statistics only; and every model was trained, evaluated on the same held-out window and refitted for a 30-day forward forecast. The LSTM was the most accurate model (RMSE ₹29.59, MAPE 1.63%).

The single most important number in Table 5 belongs to none of the four: the naive persistence forecast beat every model on the reference stock and on both validation stocks, confirming how closely daily large-cap prices approximate a random walk. The contribution of the work is therefore threefold: a fully reproducible, same-protocol comparison of four forecasting families on Indian and global equities; a quantified account of where each family gains and loses as the horizon lengthens; and a demonstration that evaluation discipline — chronological splits, training-only scaling and a persistence baseline — changes the conclusions as much as the algorithms do.

For practice, the message is equally direct. Because daily NSE large-cap equities approximate a random walk closely enough to defeat models built exclusively on historical OHLCV data, algorithmic trading strategies should not treat price-history features as a sufficient basis for forecasting: forecast skill, where it exists, must be sought in external information — order-book depth and microstructure signals, news and sentiment analysis, or macroeconomic indicators — and every deployed strategy should be benchmarked against the persistence forecast before capital is committed to it.

# References

[1] Yahoo Finance. Historical market data portal [Internet]. Available from: https://finance.yahoo.com

[2] Aroussi R. yfinance: Yahoo! Finance market data downloader [Internet]. Available from: https://pypi.org/project/yfinance/

[3] Box GEP, Jenkins GM, Reinsel GC, Ljung GM. Time Series Analysis: Forecasting and Control. 5th ed. Hoboken, NJ: Wiley; 2015.

[4] Dickey DA, Fuller WA. Distribution of the estimators for autoregressive time series with a unit root. Journal of the American Statistical Association. 1979;74(366):427–431.

[5] Fischer T, Krauss C. Deep learning with long short-term memory networks for financial market predictions. European Journal of Operational Research. 2018;270(2):654–669.

[6] Selvin S, Vinayakumar R, Gopalakrishnan EA, Menon VK, Soman KP. Stock price prediction using LSTM, RNN and CNN-sliding window model. In: International Conference on Advances in Computing, Communications and Informatics (ICACCI); 2017. p. 1643–1647.

[7] Siami-Namini S, Tavakoli N, Namin AS. A comparison of ARIMA and LSTM in forecasting time series. In: 17th IEEE International Conference on Machine Learning and Applications (ICMLA); 2018. p. 1394–1401.

[8] Patel J, Shah S, Thakkar P, Kotecha K. Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. Expert Systems with Applications. 2015;42(1):259–268.

[9] Taylor SJ, Letham B. Forecasting at scale. The American Statistician. 2018;72(1):37–45.

[10] National Stock Exchange of India [Internet]. Available from: https://www.nseindia.com

[11] BSE India [Internet]. Available from: https://www.bseindia.com

[12] Seabold S, Perktold J. statsmodels: Econometric and statistical modeling with Python. In: Proceedings of the 9th Python in Science Conference (SciPy); 2010. p. 92–96.

[13] Hochreiter S, Schmidhuber J. Long short-term memory. Neural Computation. 1997;9(8):1735–1780.

[14] Chen T, Guestrin C. XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining; 2016. p. 785–794.

[15] Abadi M, Barham P, Chen J, et al. TensorFlow: A system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI); 2016. p. 265–283.

[16] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research. 2011;12:2825–2830.

[17] Krauss C, Doering A, Jonigk S. Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. European Journal of Operational Research. 2017;259(2):689–702.